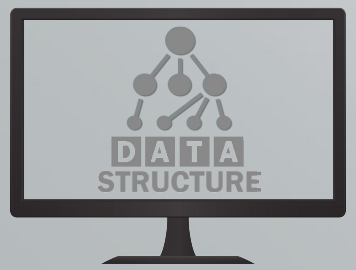
Arab Academy for Science , Technology & Maritime Transport
College of Computing and Information Technology

Data Structures and Algorithms (CS212)



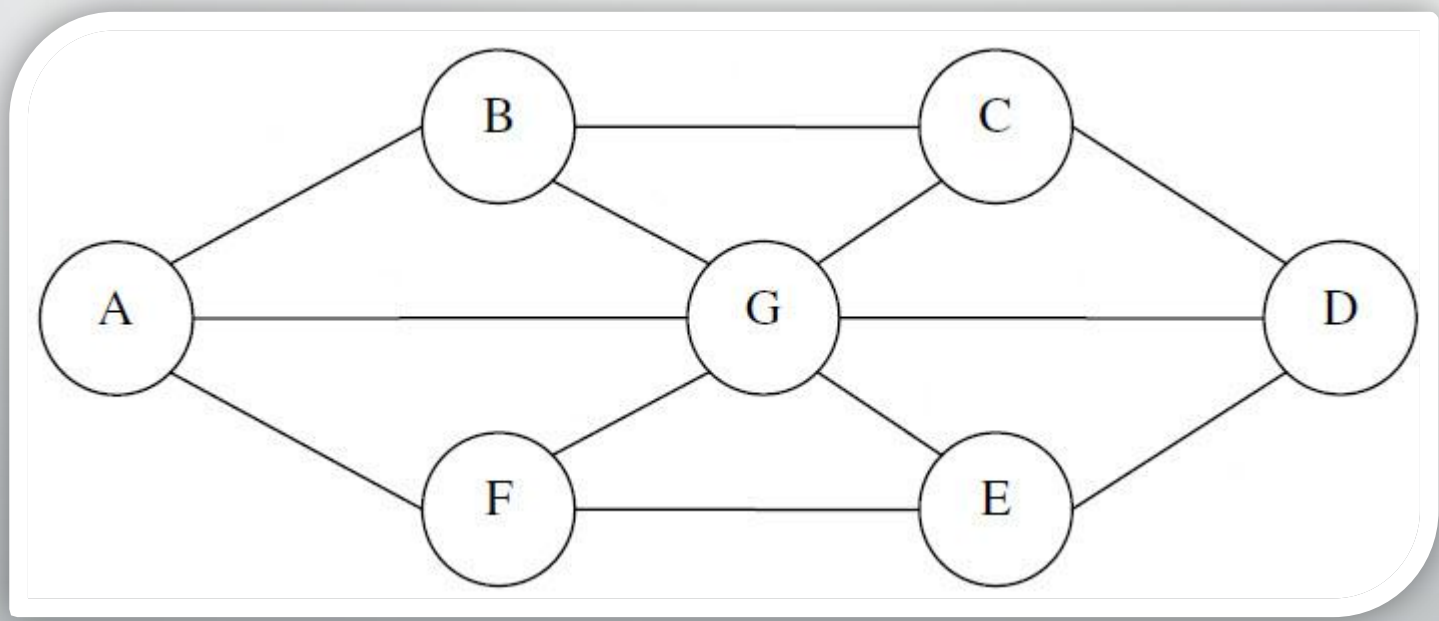
Section 08 : Graphs

Section Index

- What is a Graph ?
- Graph as a Mathematical Definition
- Graph Edge Types
- Weighted vs Unweighted Graph
- Graph Properties
- Graph Representation: Matrix/List
- Graph Traversal: BFS/DFS

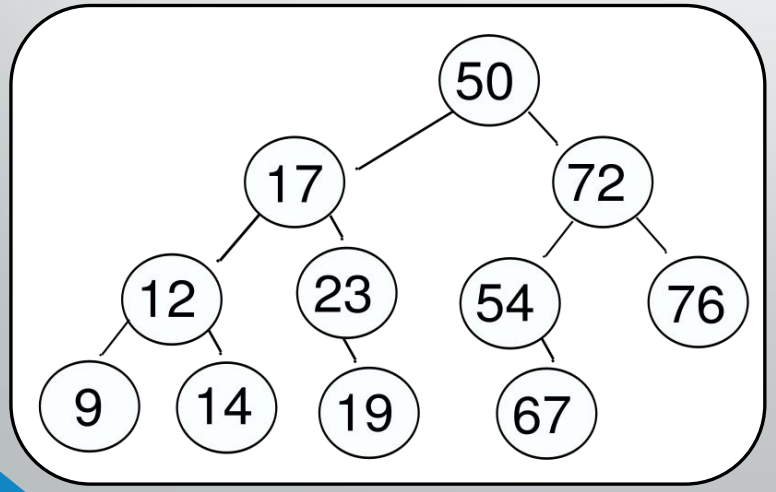
Graphs

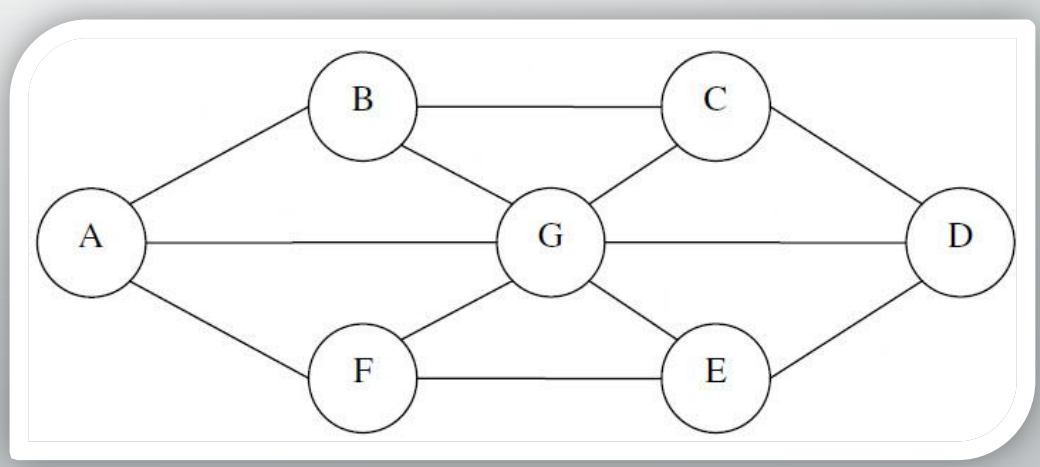
- ✔ Graph is a collection of nodes (vertices) that are connected to each other through a set of edges.
- ✔ edges can connect nodes in any possible way.



Graphs

- ✔ Actually Tree is a special kind of graph , but all nodes must be reachable from the root and there must be exactly one possible path from root
- ✔ In graphs , there are no rules , we will talk about how you will represent it in coming slides ...





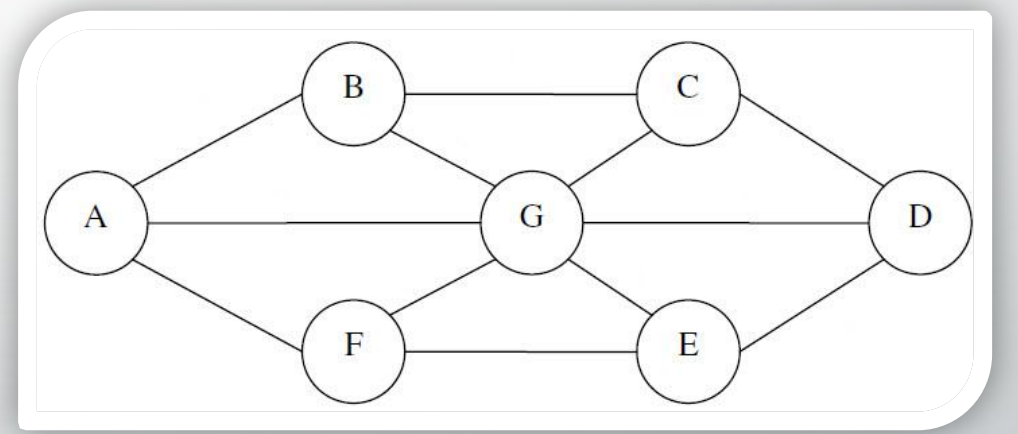
Graph (Mathematical Definition)

- A Graph **G** is an ordered pair of a set **V** of vertices and a set **E** of edges

  G = (V , E)

- Ordered pair :
  (a , b) != (b , a)

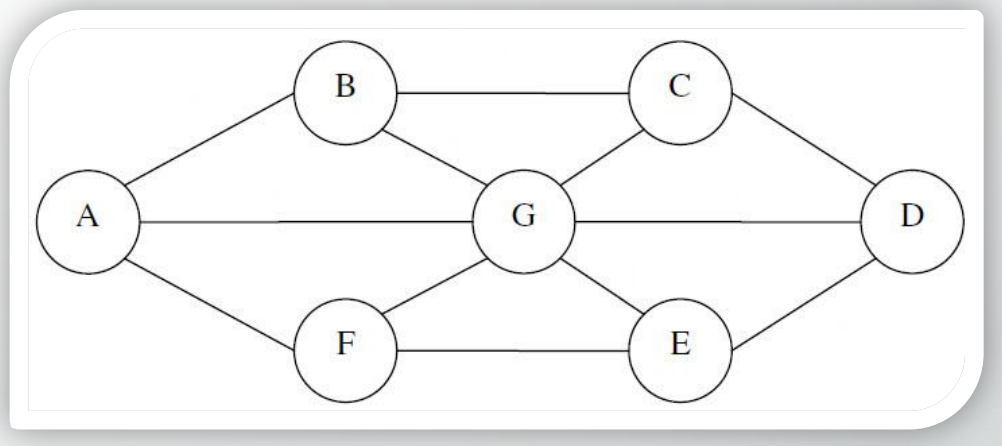
- Unordered pair :
  { a , b } = { b , a }



Graph (Mathematical Definition)

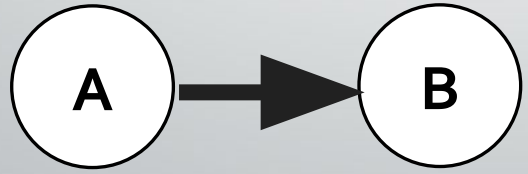
- A Graph **G** is an ordered pair of a set **V** of vertices and a set **E** of edges

G = (V , E)



- Edge types :

1- Directed Edge



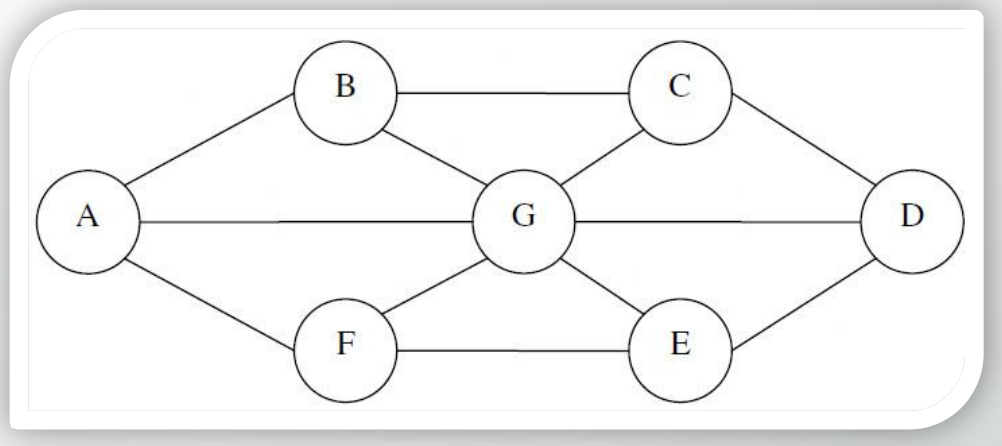
(a , b)

2- Undirected Edge

{ a , b }

Graph (Mathematical Definition)

- A Graph **G** is an ordered pair of a set **V** of vertices and a set **E** of edges



G = (V , E)

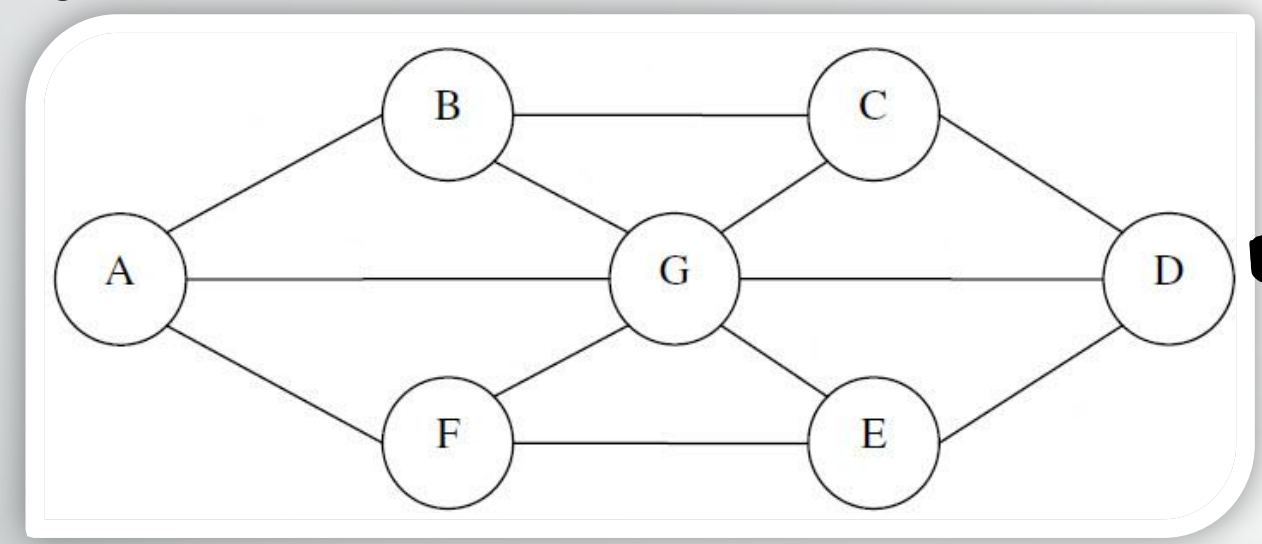
- **Vertices :**

  V = { A , B , C , D , E , F , G }

- **Edges:**

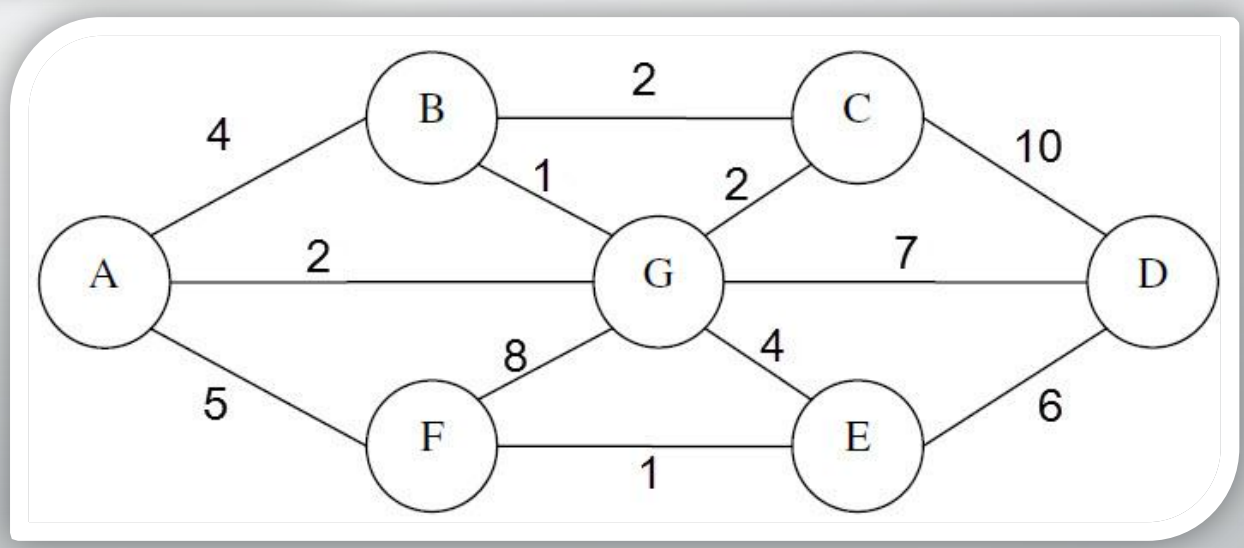
  E = { {A,B},{A,G},{A,F},{B,C},{B,G},{C,G},{C,D},
  {G,F},{G,E},{G,D},{D,E},{F,E} }

Graph (Weighted vs unweighted)



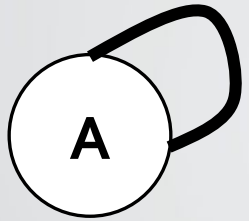
✔ Unweighted Graph

✔ Weighted Graph

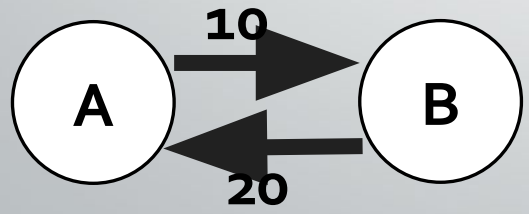


Graph (Important Properties)

- Self Loop



- Multi-edge



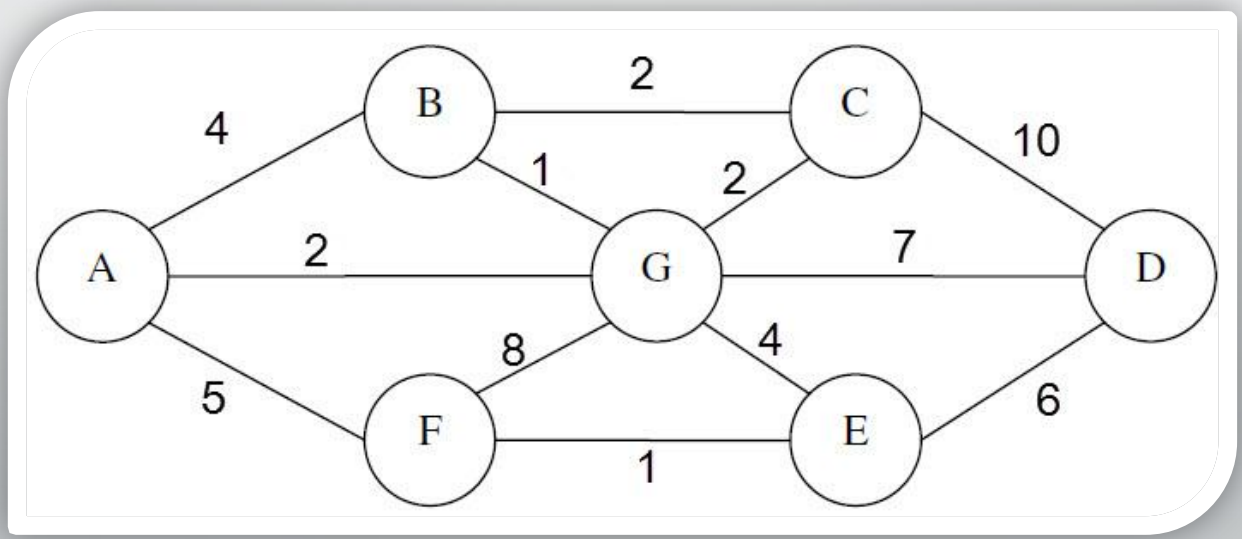
- If there's no self loops or multi-edges , graph will be called a **simple graph**

- Number of Edges **|E|** in a simple graph is :
  - 0 <= |E| <= n(n-1) directed
  - 0 <= |E| <= n(n-1)/2 undirected

Graph Representation (Implementation)

✔ **Graphs can be represented by :**

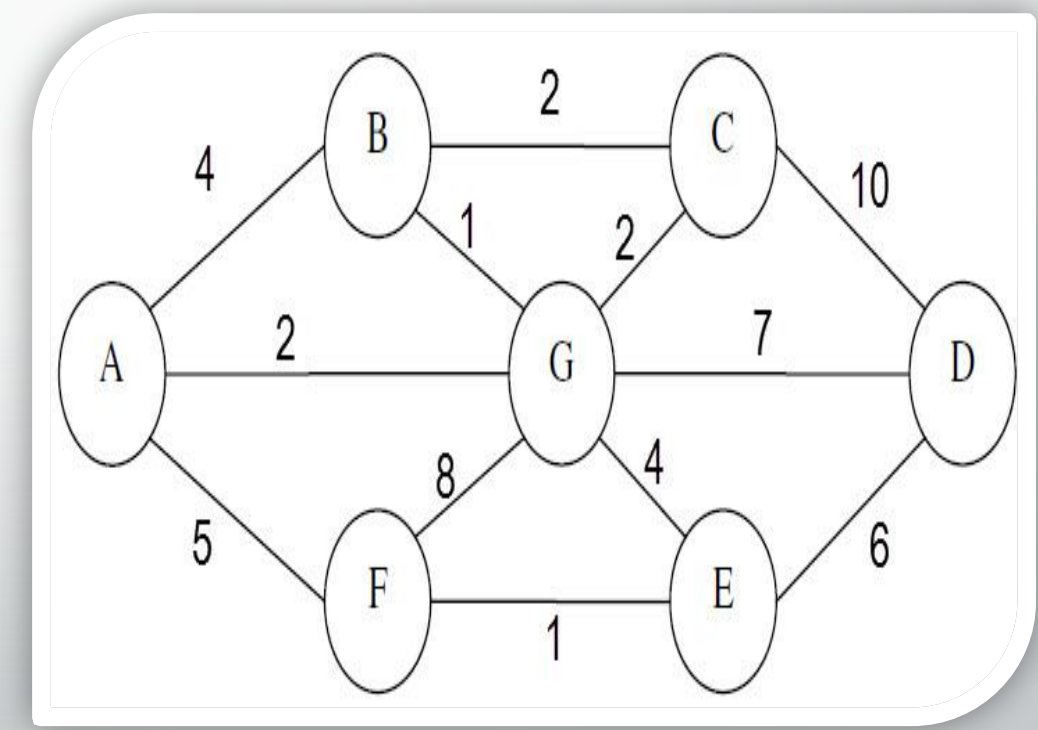
- 2D Arrays **(Adjacency Matrix)**
- 1D Array + Linked Lists **(Adjacency List)**



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

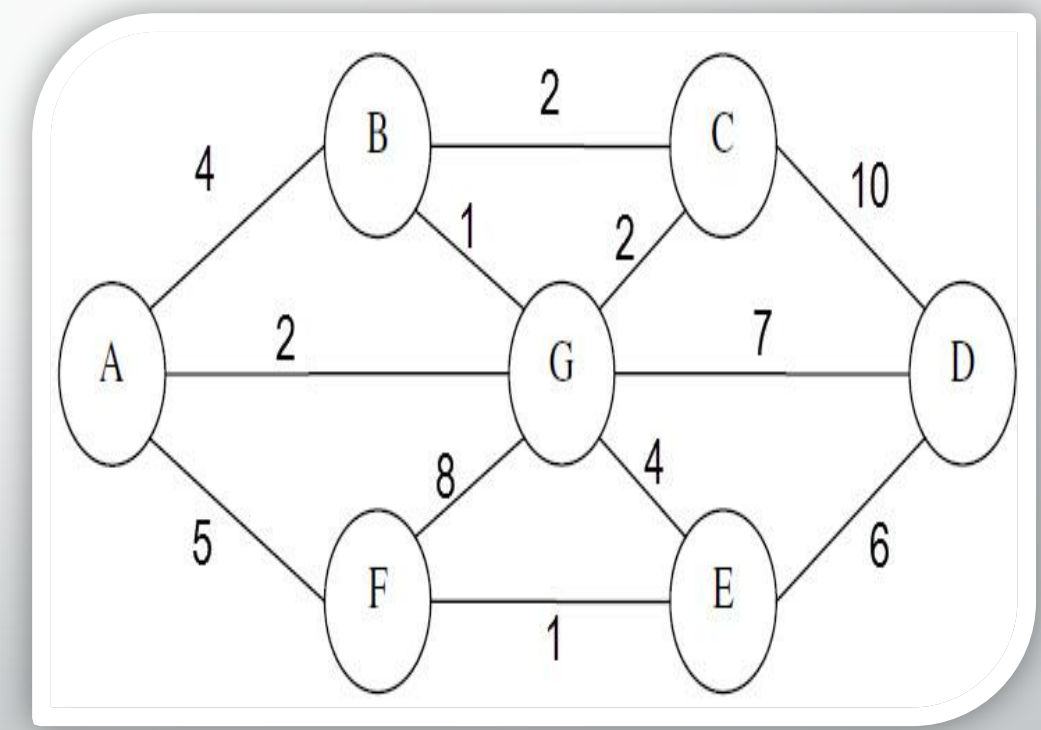
	A	B	C	D	E	F	G
A							
B							
C							
D							
E							
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

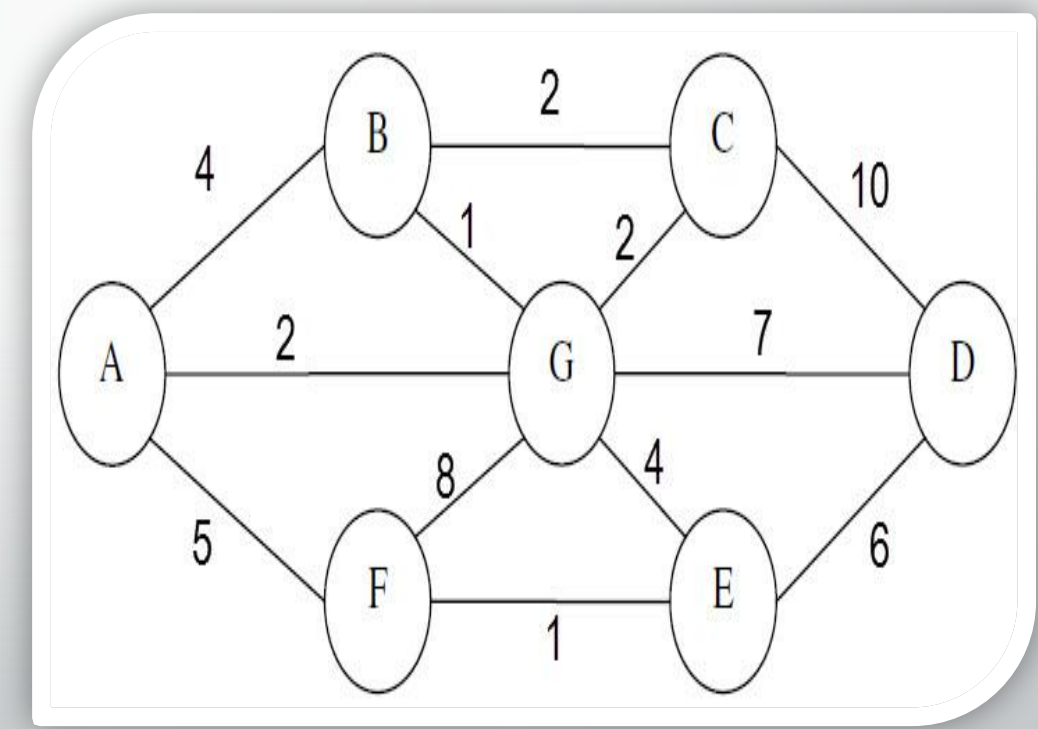
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B							
C							
D							
E							
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

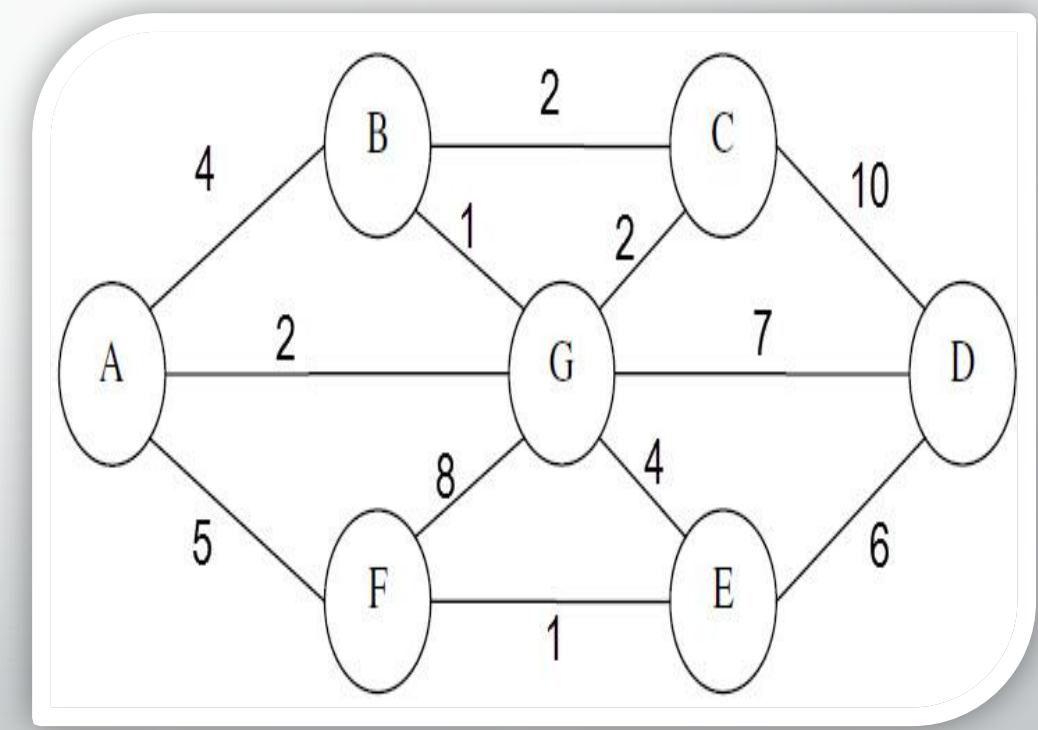
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C							
D							
E							
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

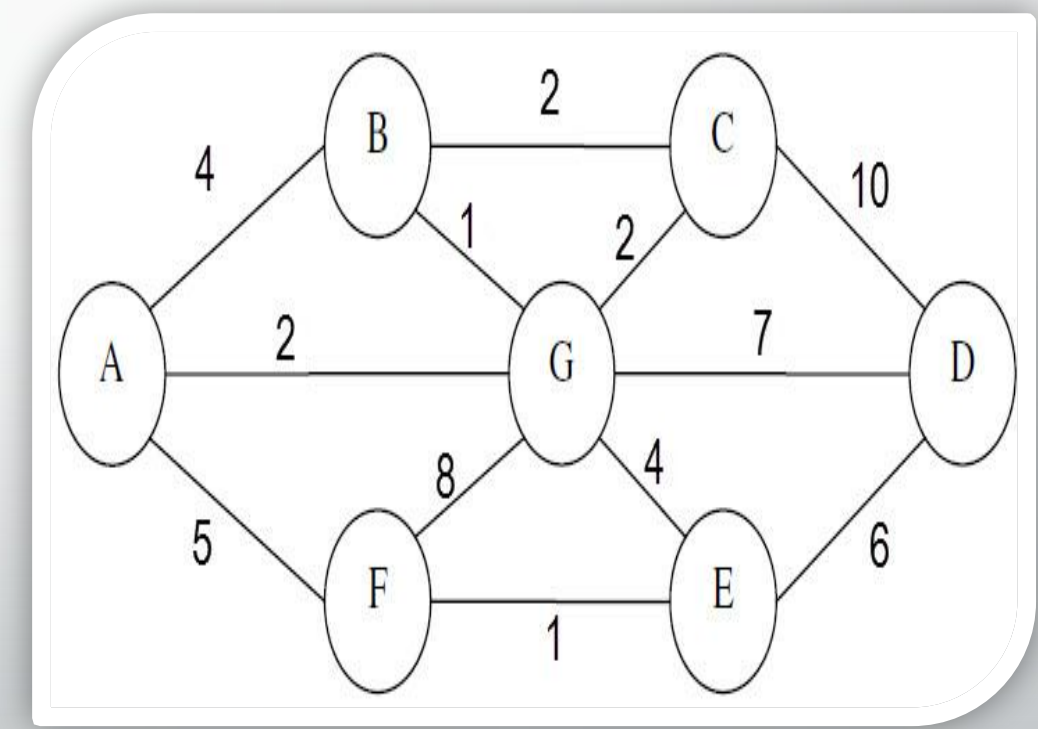
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D							
E							
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

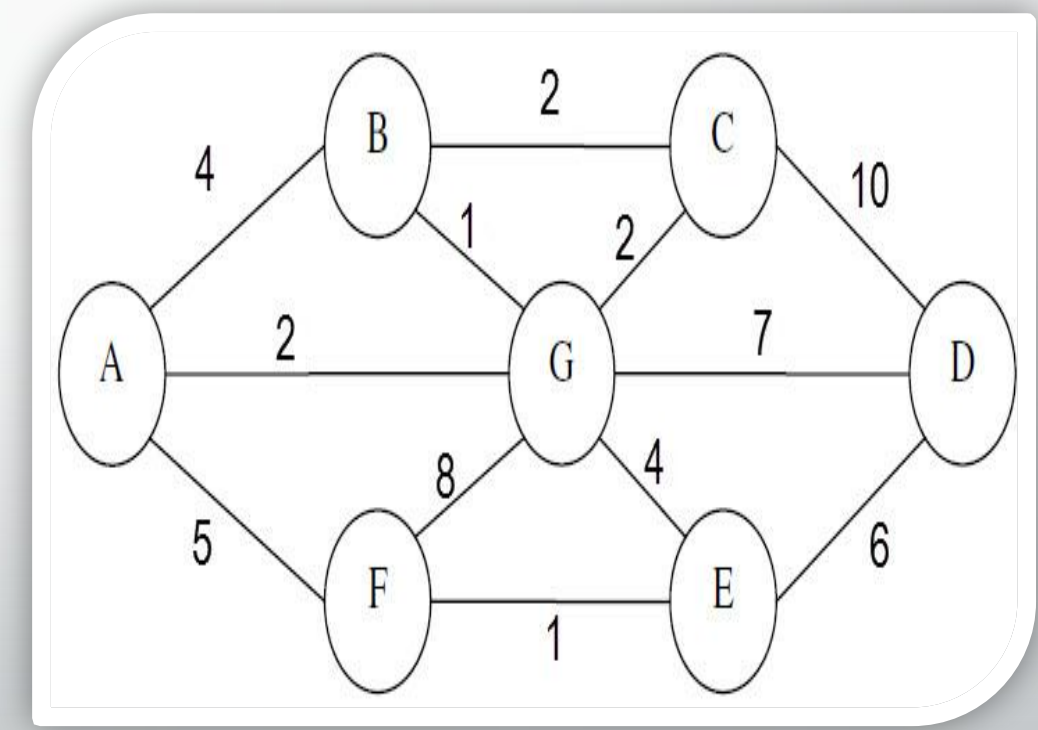
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D	0	0	10	0	6	0	7
E							
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

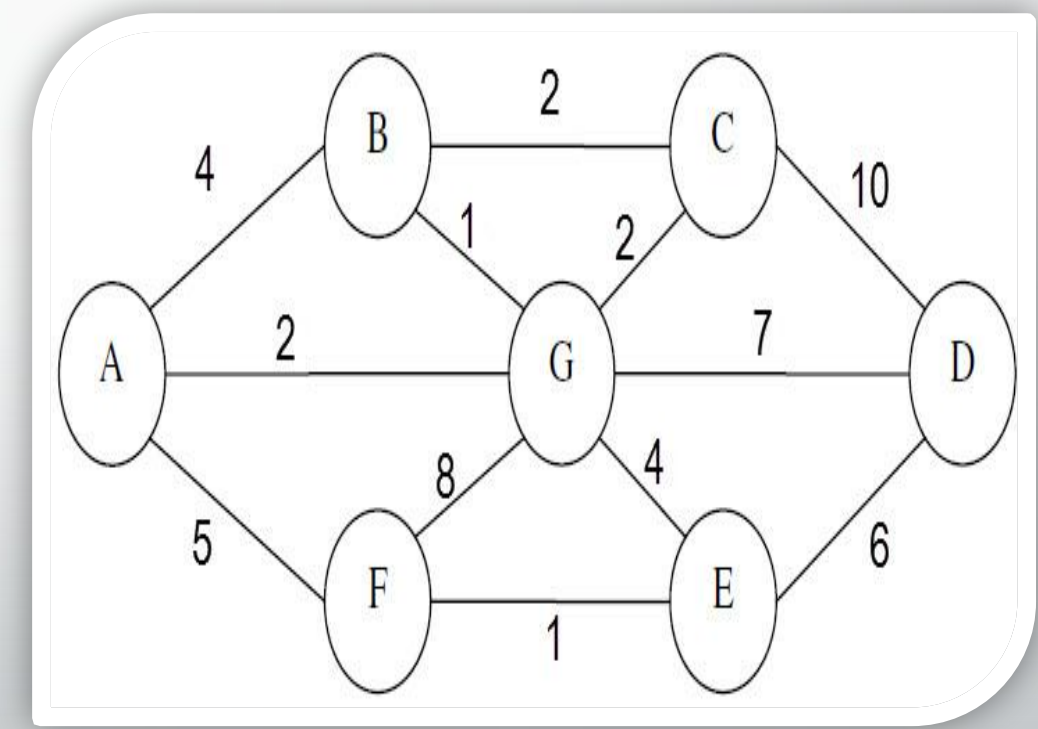
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D	0	0	10	0	6	0	7
E	0	0	0	6	0	1	4
F							
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

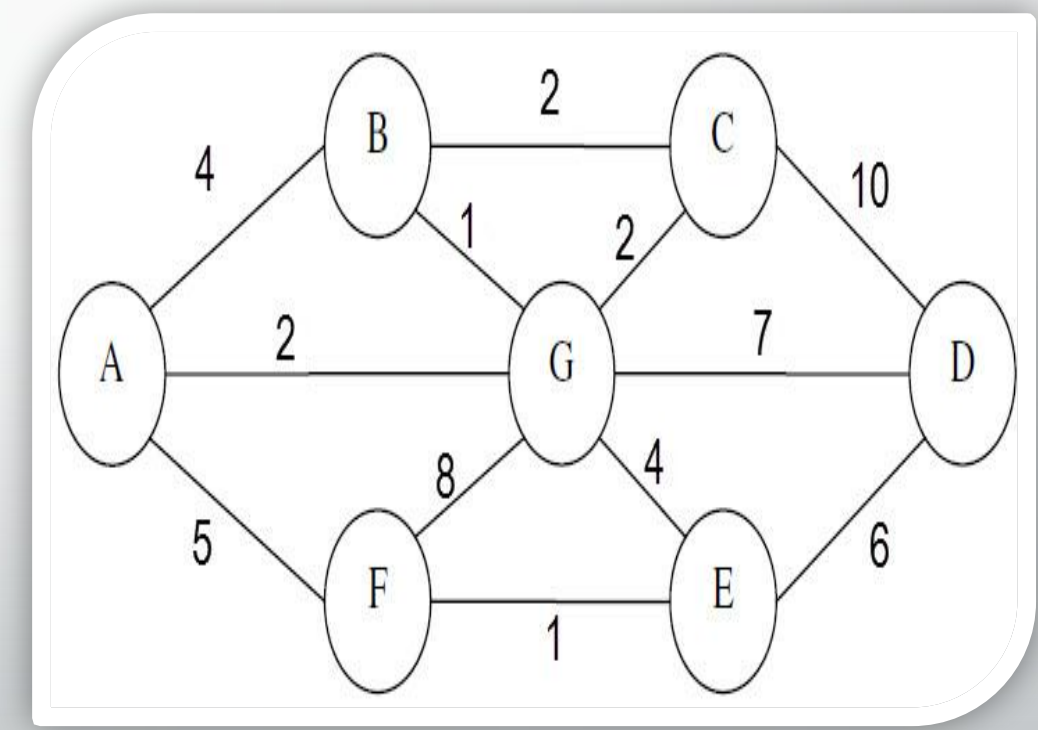
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D	0	0	10	0	6	0	7
E	0	0	0	6	0	1	4
F	5	0	0	0	1	0	8
G							



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

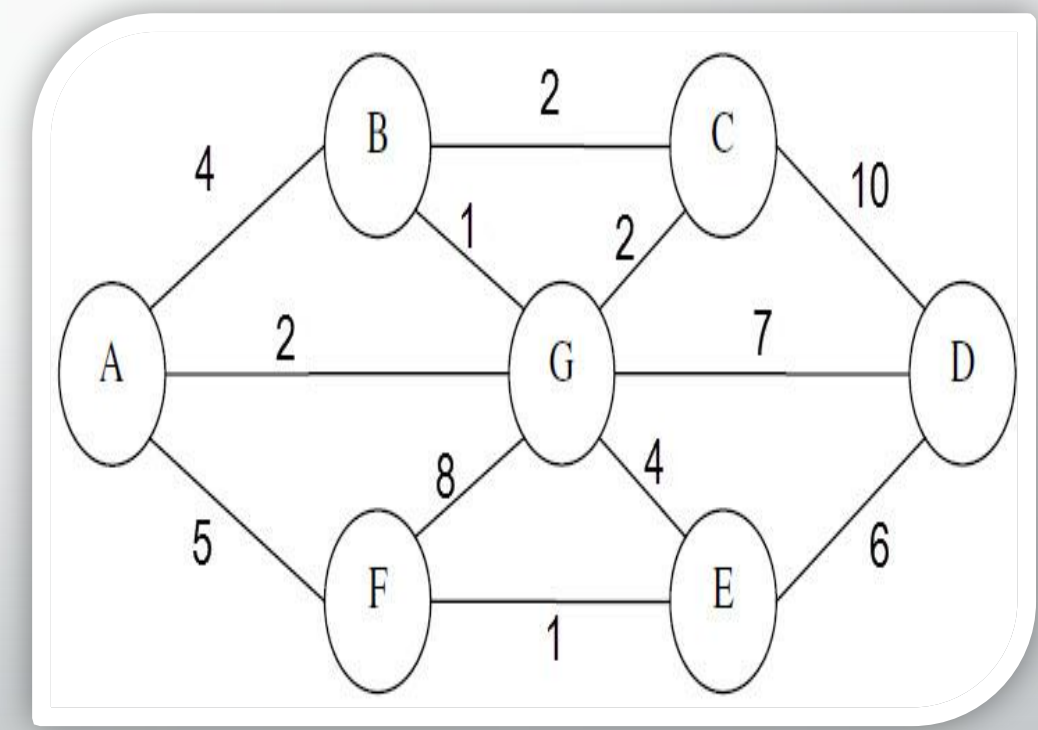
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D	0	0	10	0	6	0	7
E	0	0	0	6	0	1	4
F	5	0	0	0	1	0	8
G	2	1	2	7	4	8	0



Graph Representation

- 2D Arrays **(Adjacency Matrix)**

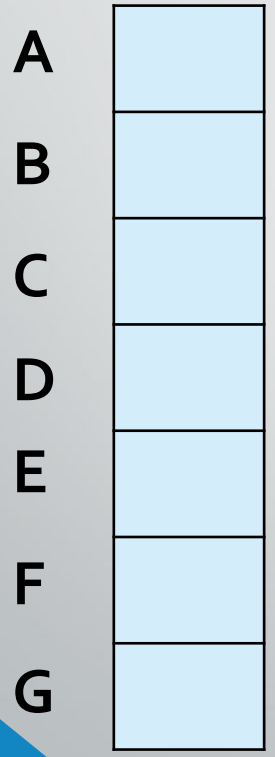
	A	B	C	D	E	F	G
A	0	4	0	0	0	5	2
B	4	0	2	0	0	0	1
C	0	2	0	10	0	0	2
D	0	0	10	0	6	0	7
E	0	0	0	6	0	1	4
F	5	0	0	0	1	0	8
G	2	1	2	7	4	8	0



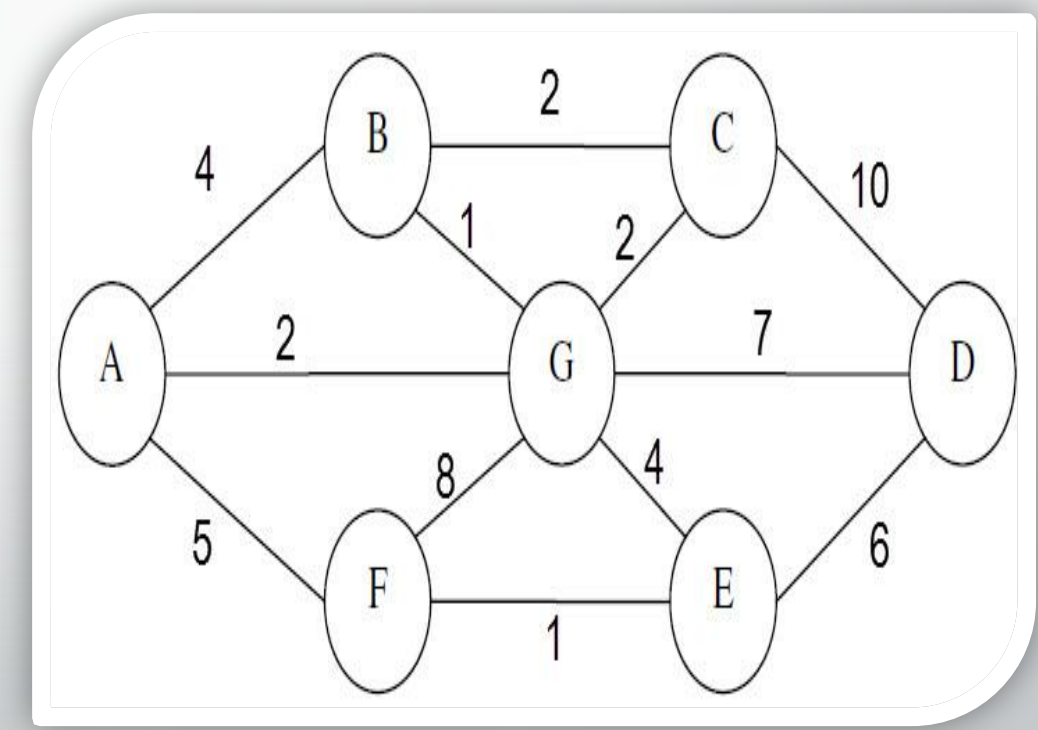
- Matrix is symmetric because it's undirected

Graph Representation

- 1D Array + Linked Lists **(Adjacency List)**

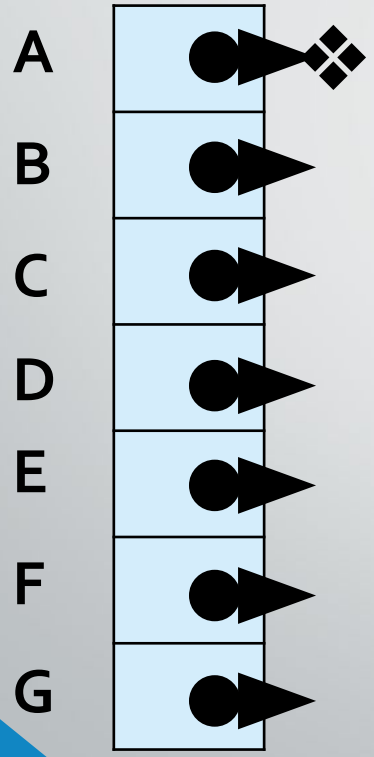


- First , Create a 1D Array of size (# of Vertices)

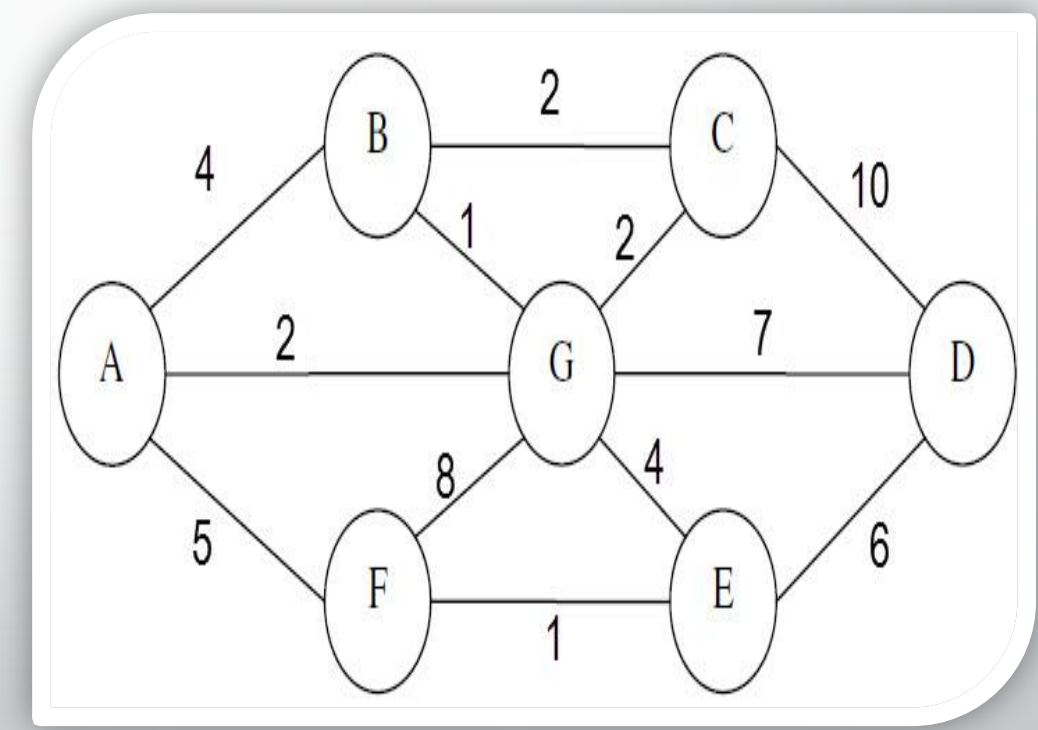


Graph Representation

- 1D Array + Linked Lists **(Adjacency List)**

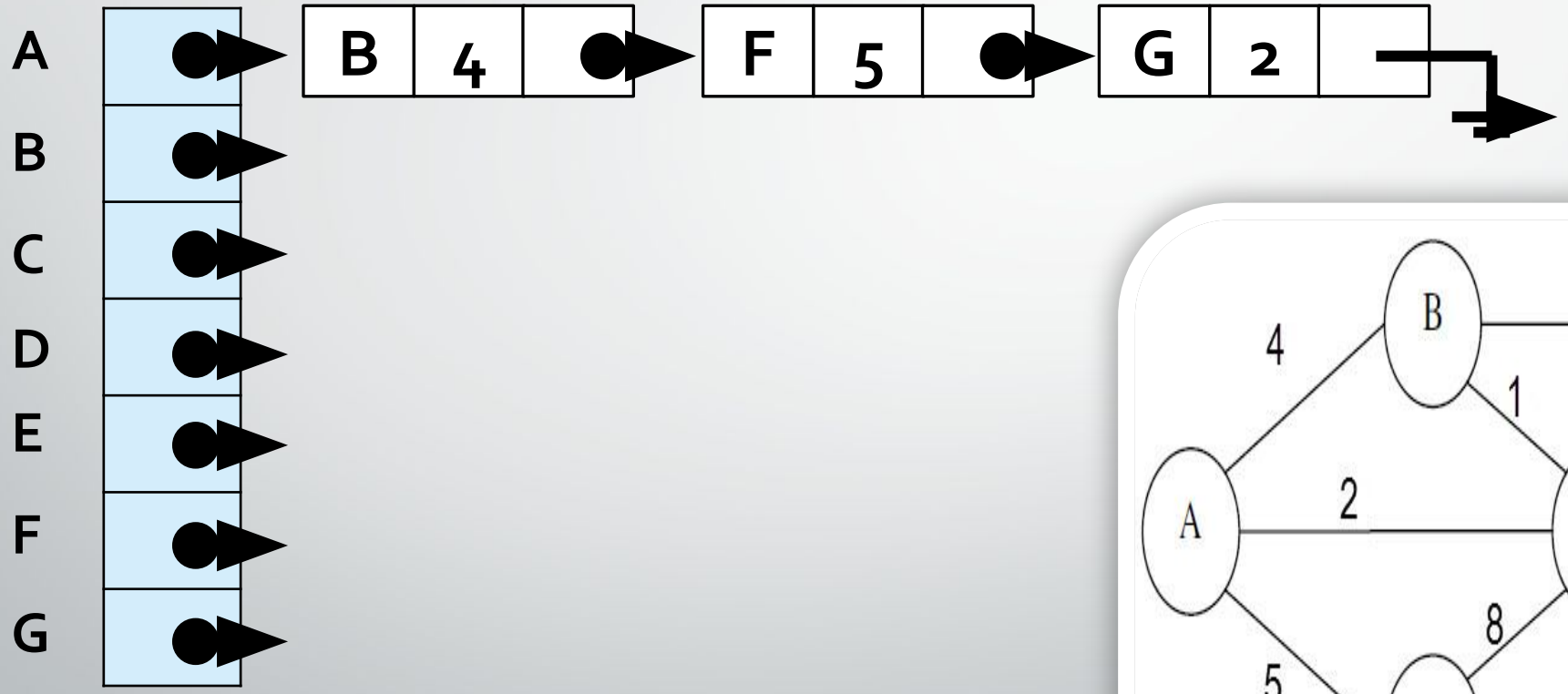


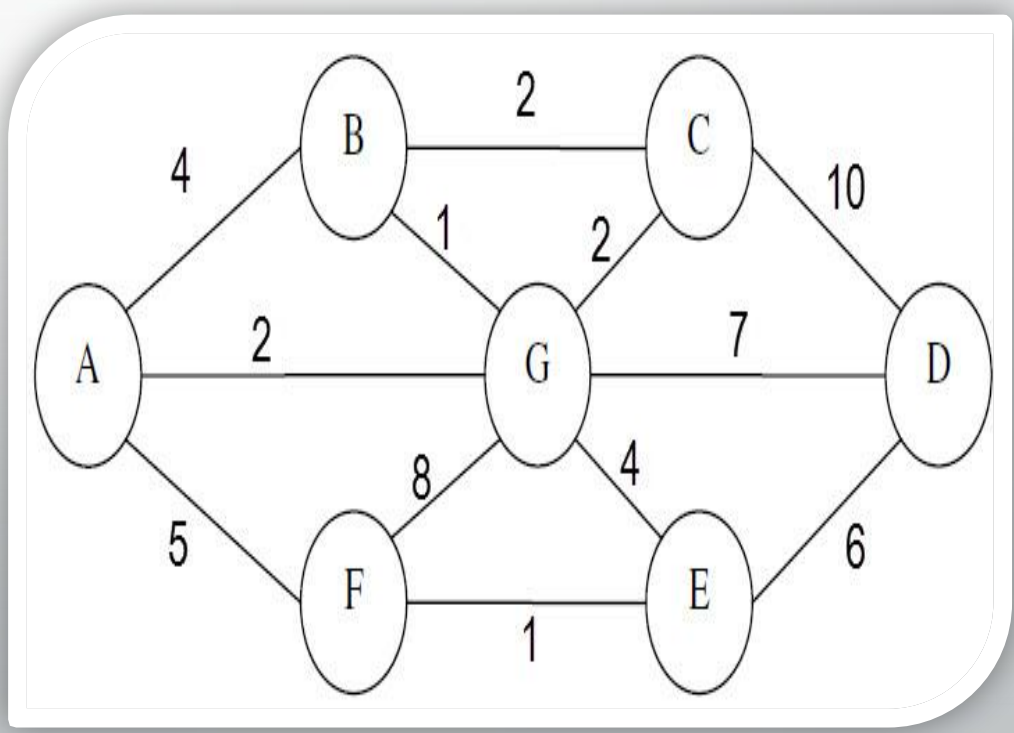
For each element in the array it will contain a pointer to linked list of it's connected nodes



Graph Representation

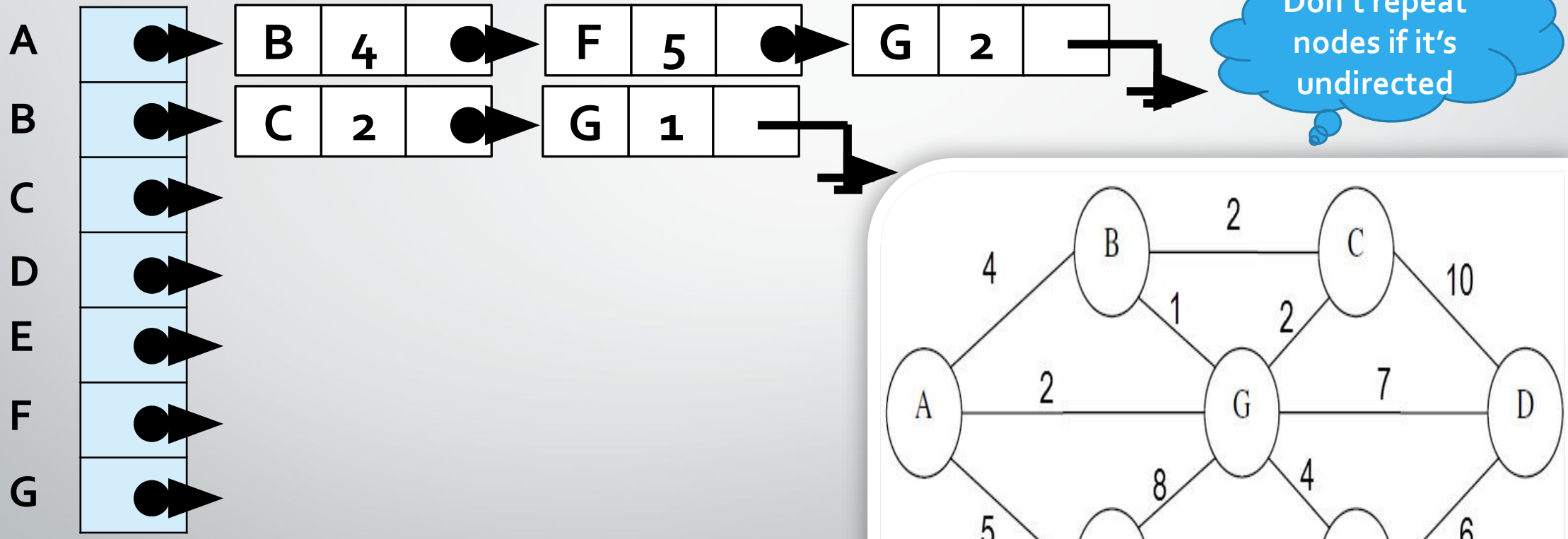
- 1D Array + Linked Lists **(Adjacency List)**





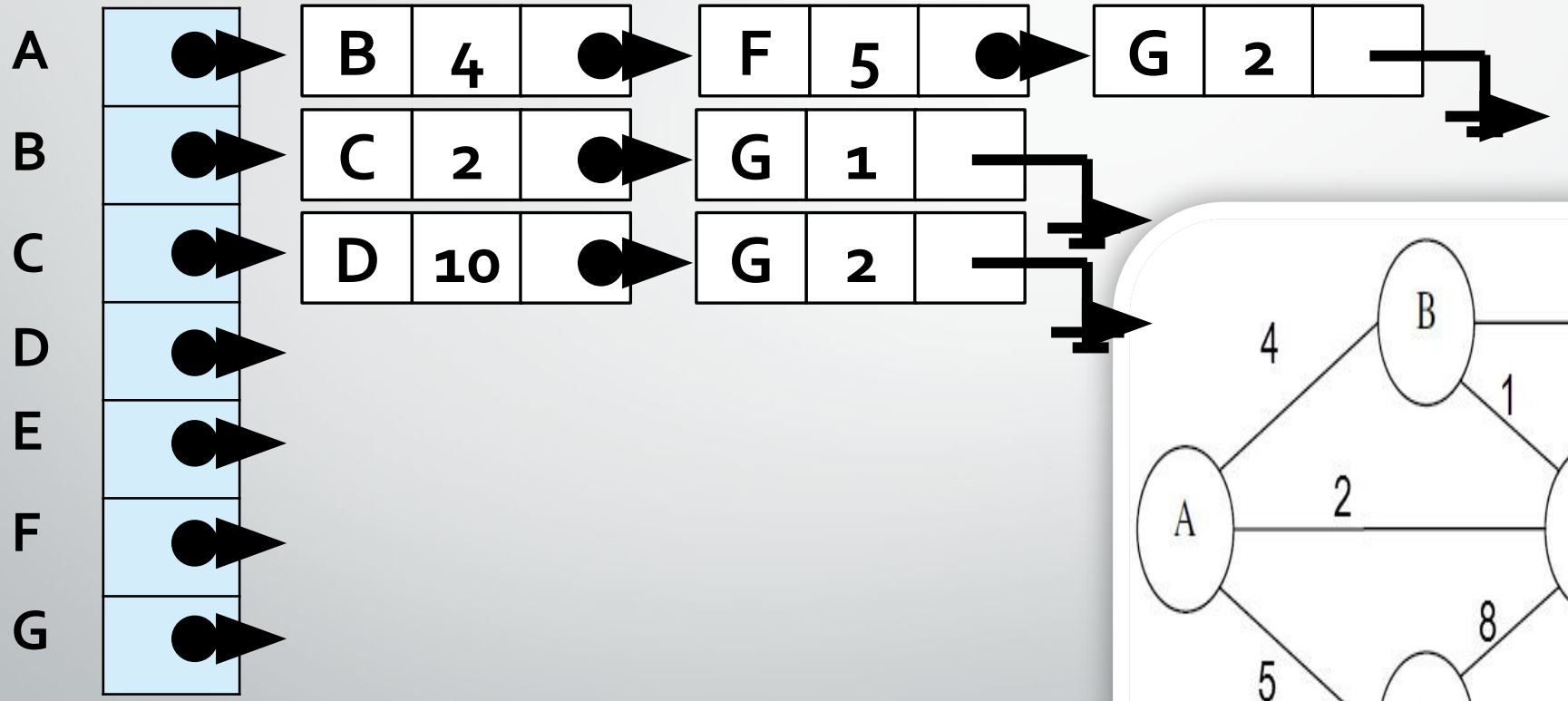
Graph Representation

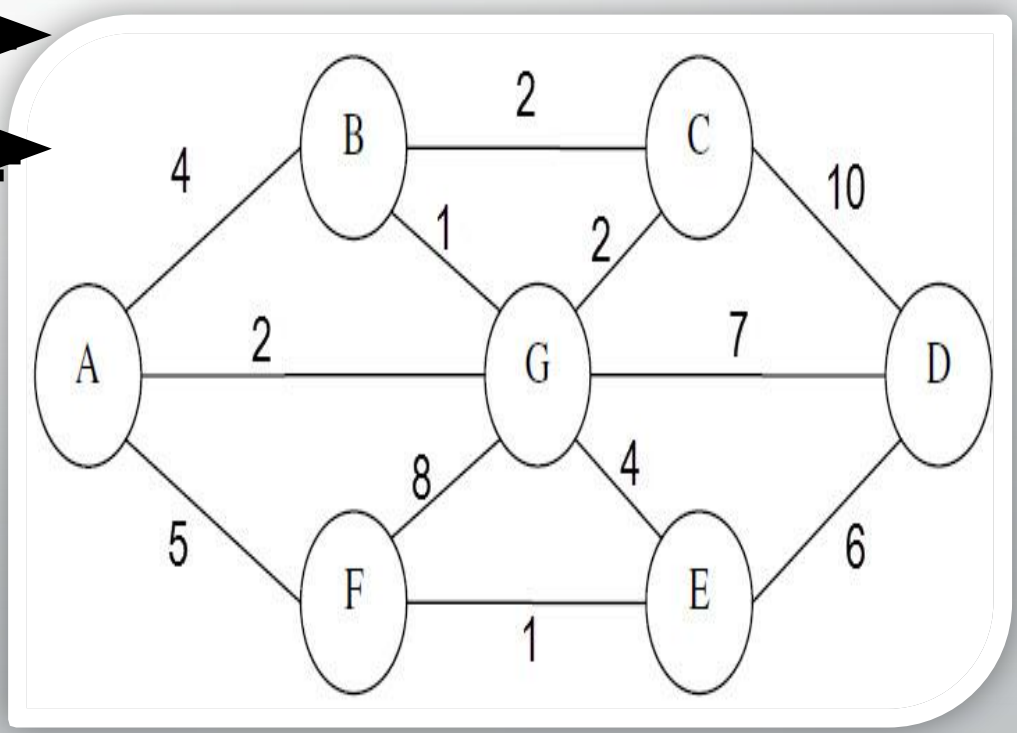
- 1D Array + Linked Lists **(Adjacency List)**



Graph Representation

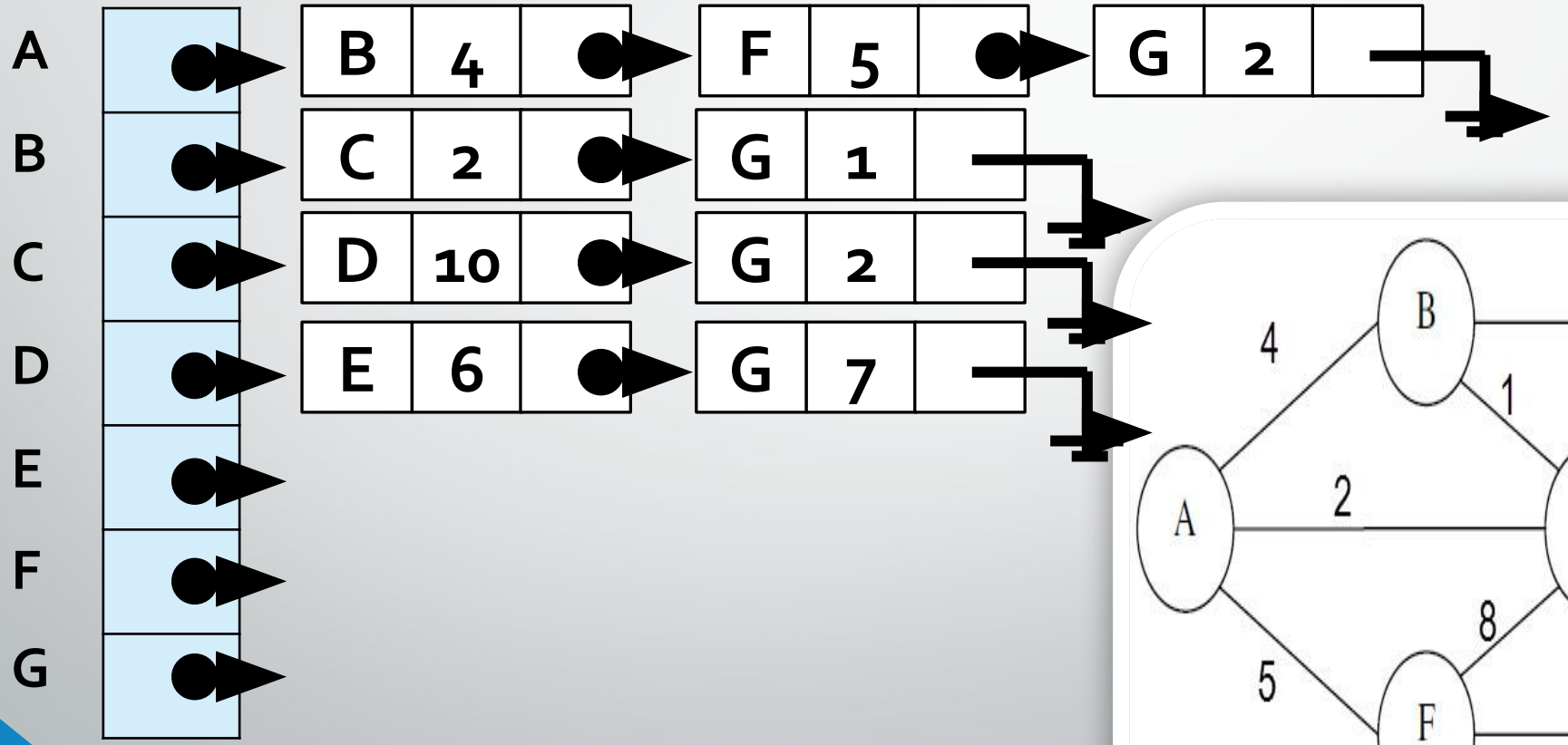
- 1D Array + Linked Lists **(Adjacency List)**

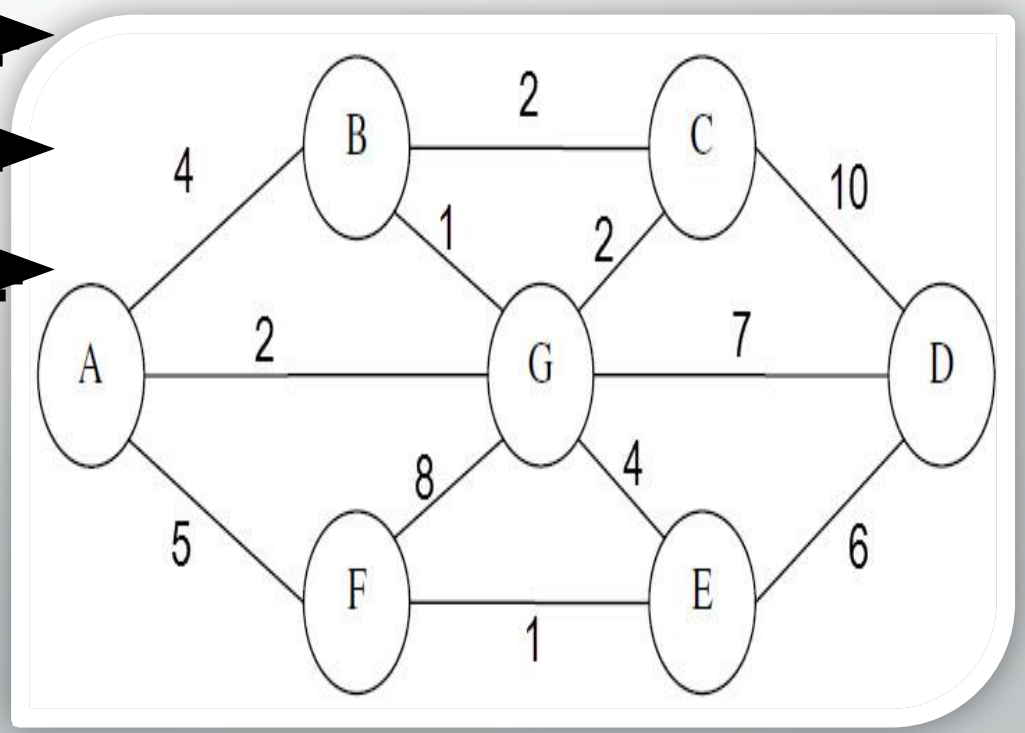




Graph Representation

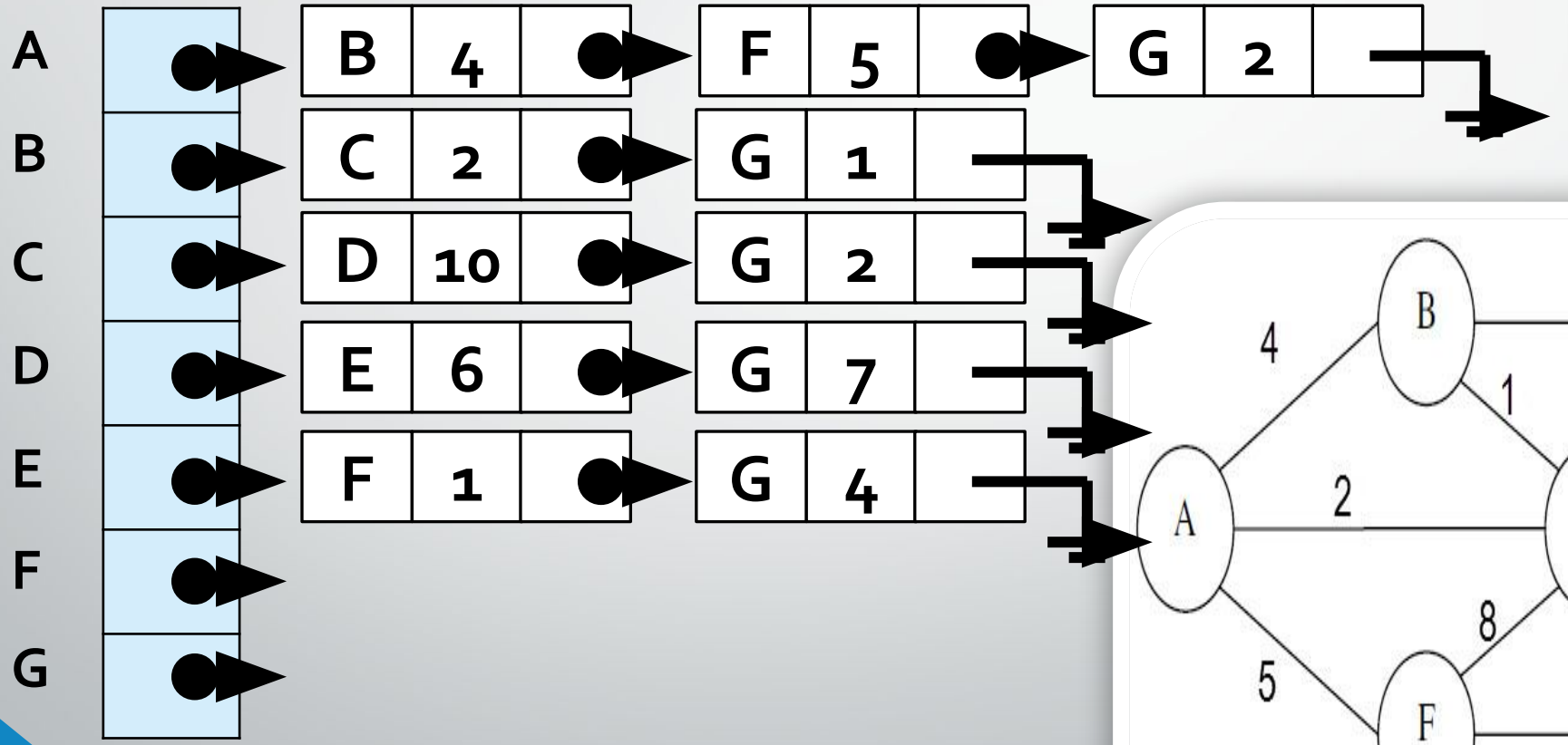
- 1D Array + Linked Lists **(Adjacency List)**

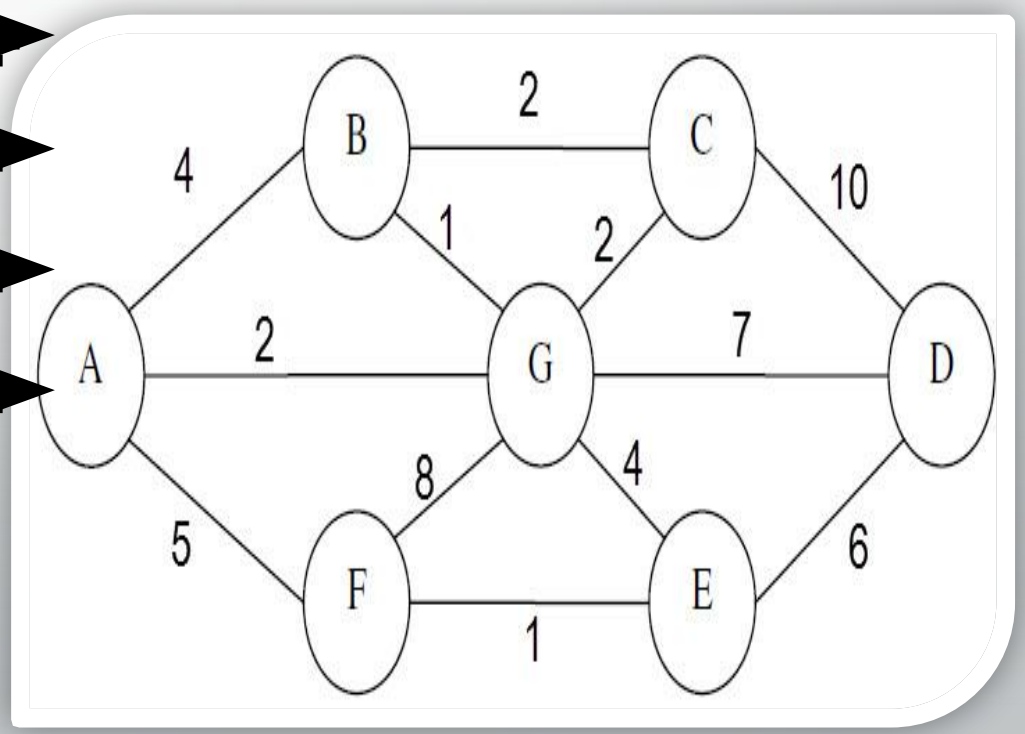




Graph Representation

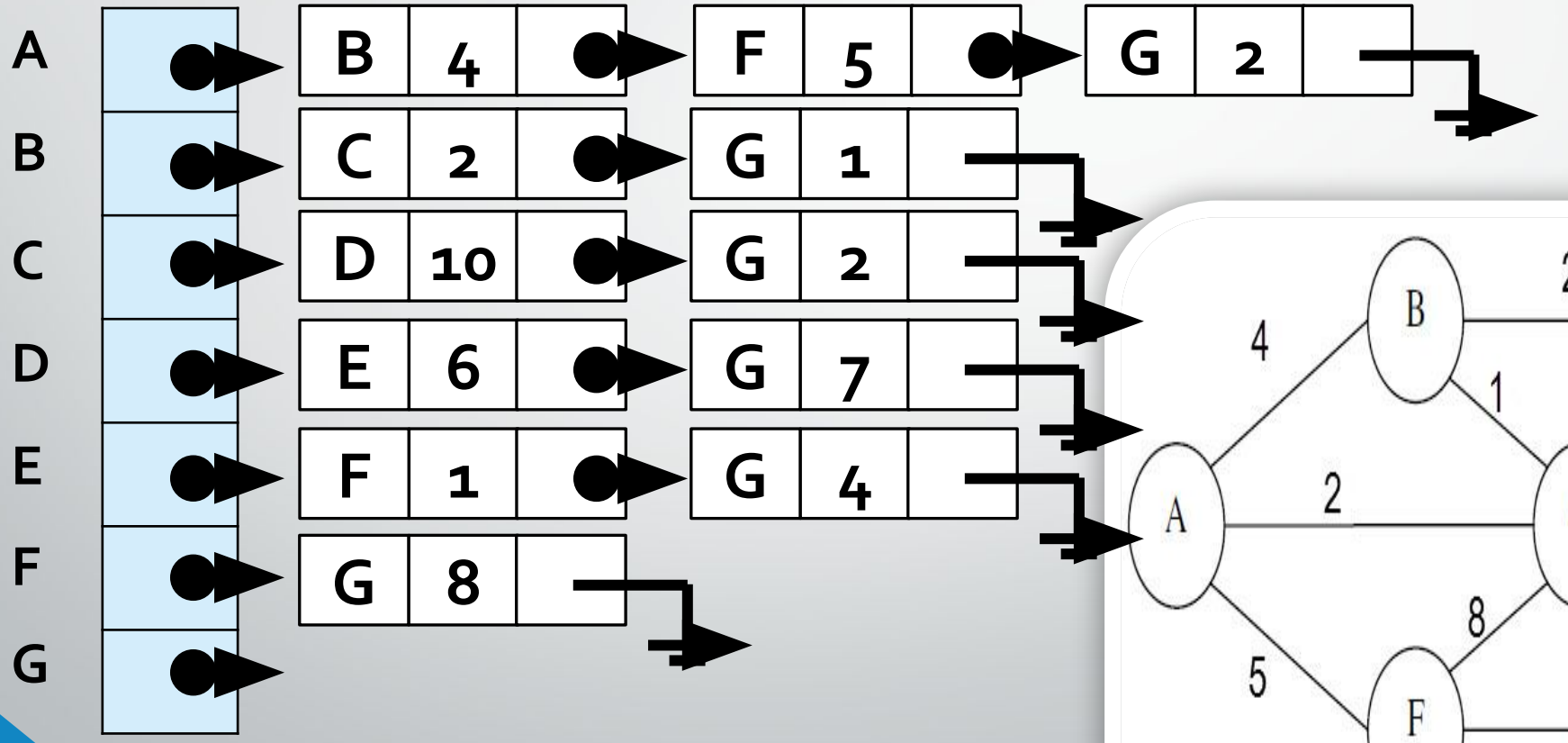
- 1D Array + Linked Lists **(Adjacency List)**

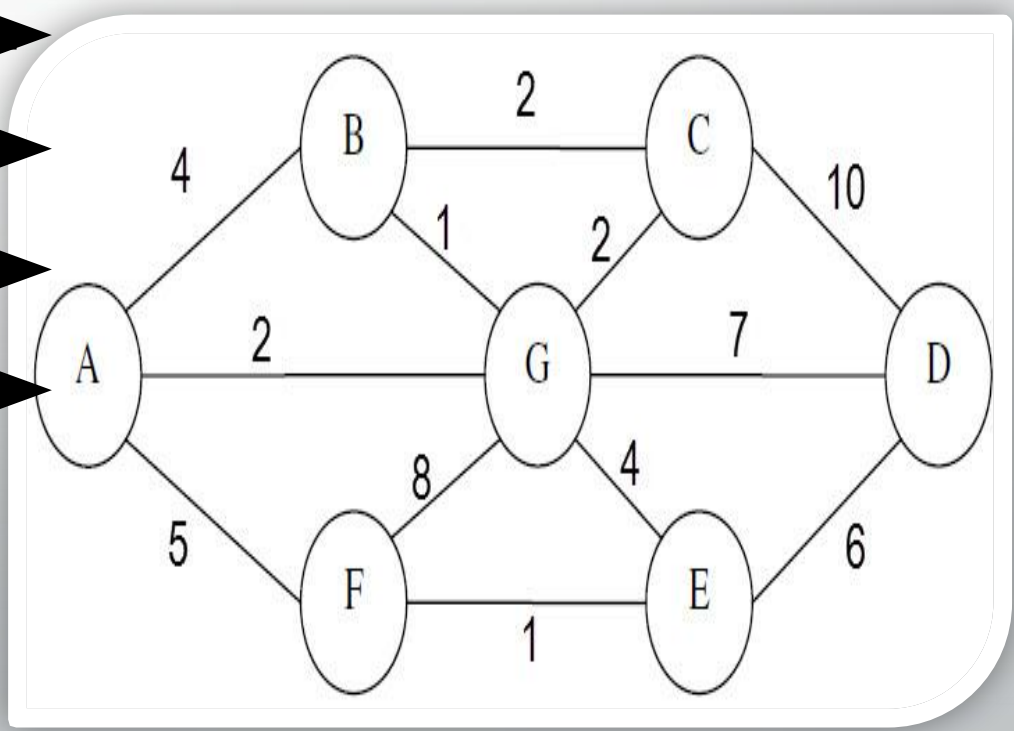




Graph Representation

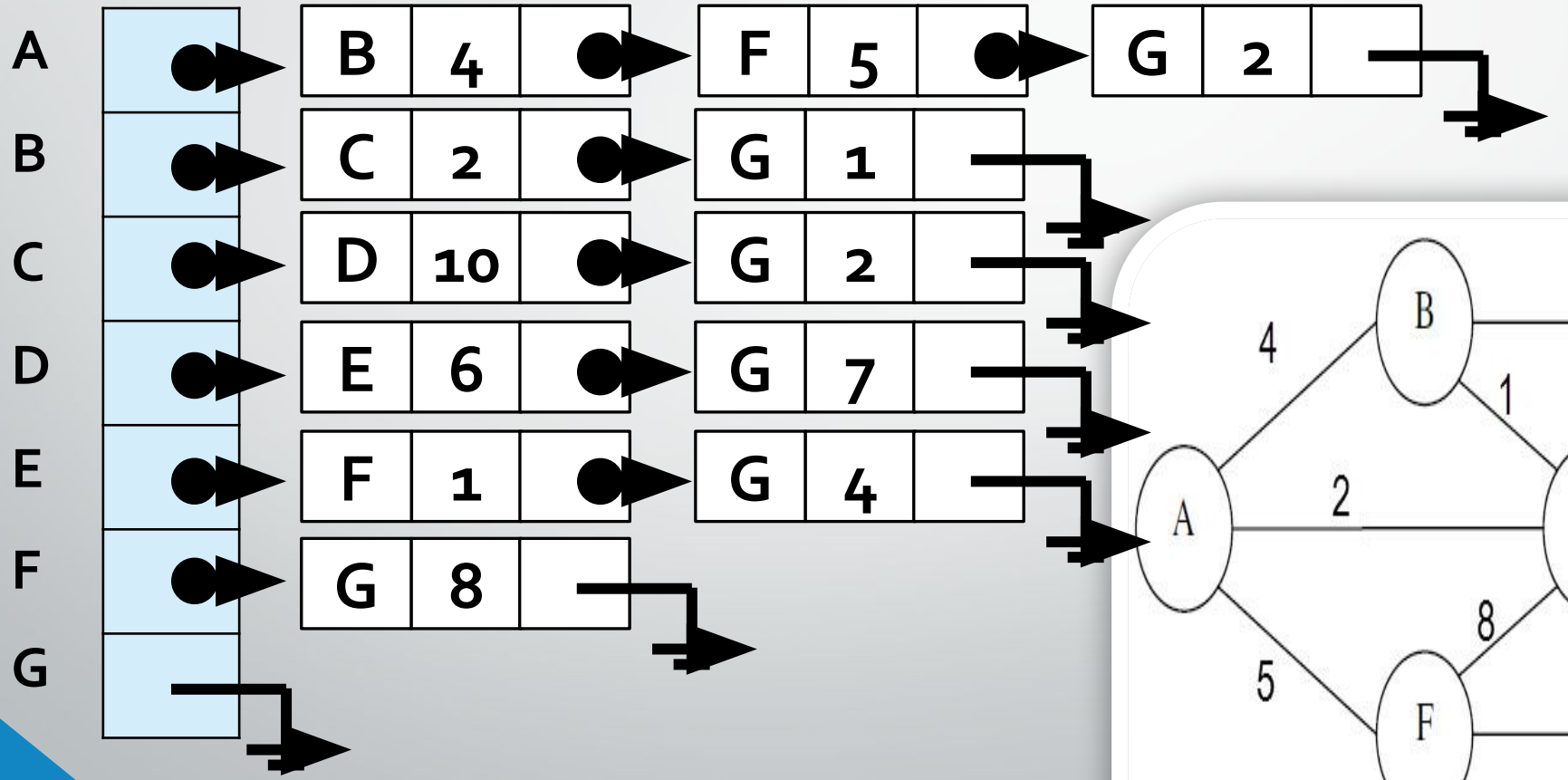
- 1D Array + Linked Lists **(Adjacency List)**

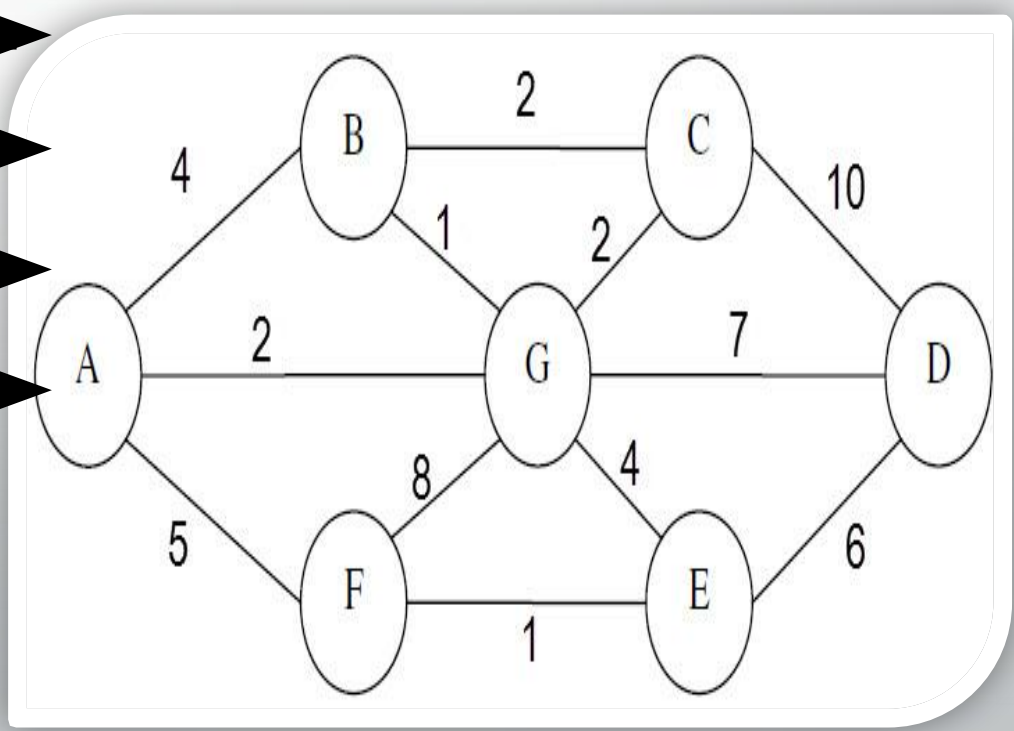




Graph Representation

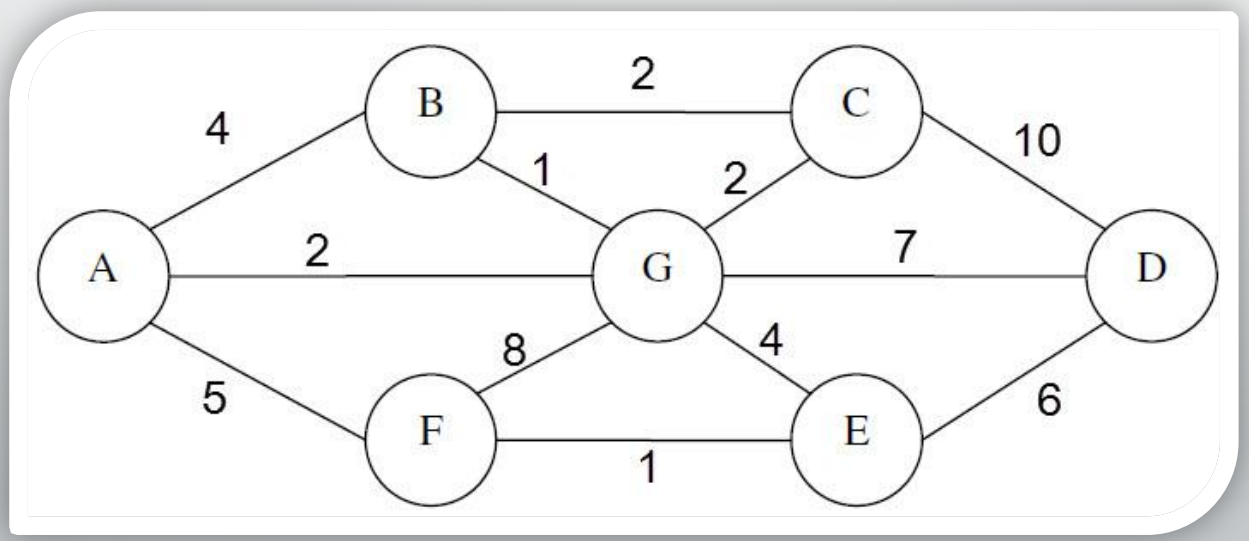
- 1D Array + Linked Lists **(Adjacency List)**





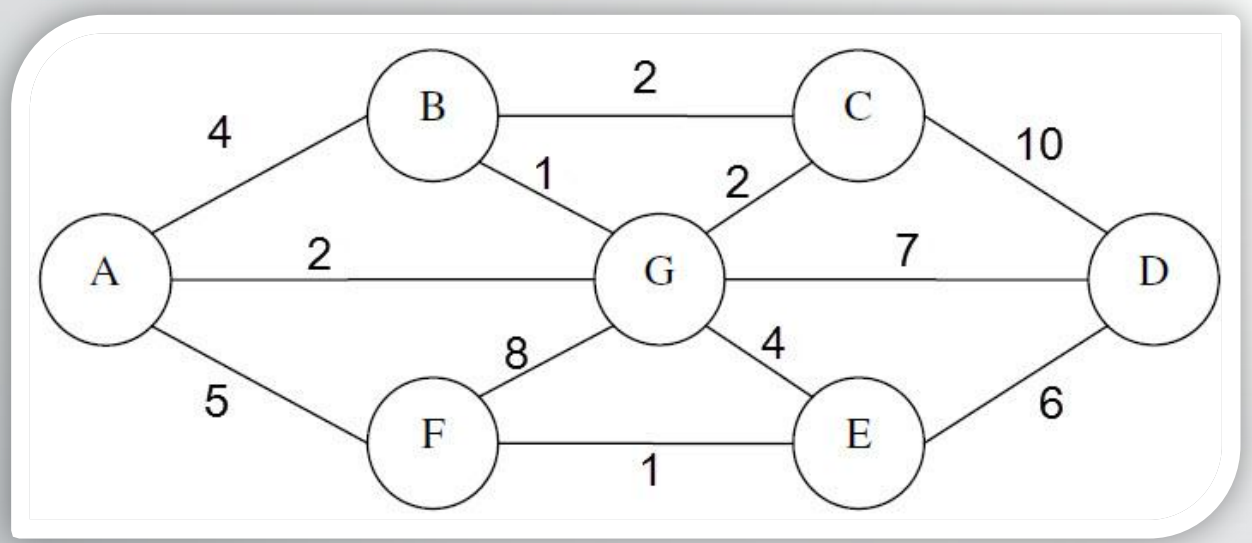
Graph Traversal

- ✔ Breadth First Search (BFS)
- ✔ Depth First Search (DFS)



Graph Traversal

- ✔ Breadth First Search (BFS)

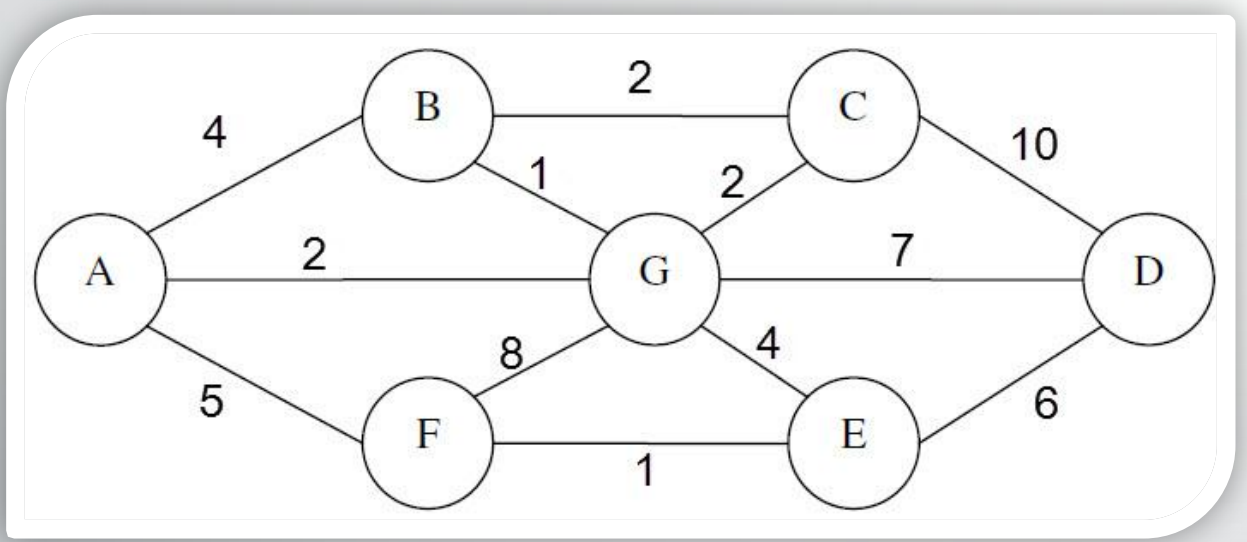


- ✔ First , create a queue

- ✔ Result:

Graph Traversal

- Breadth First Search (BFS)



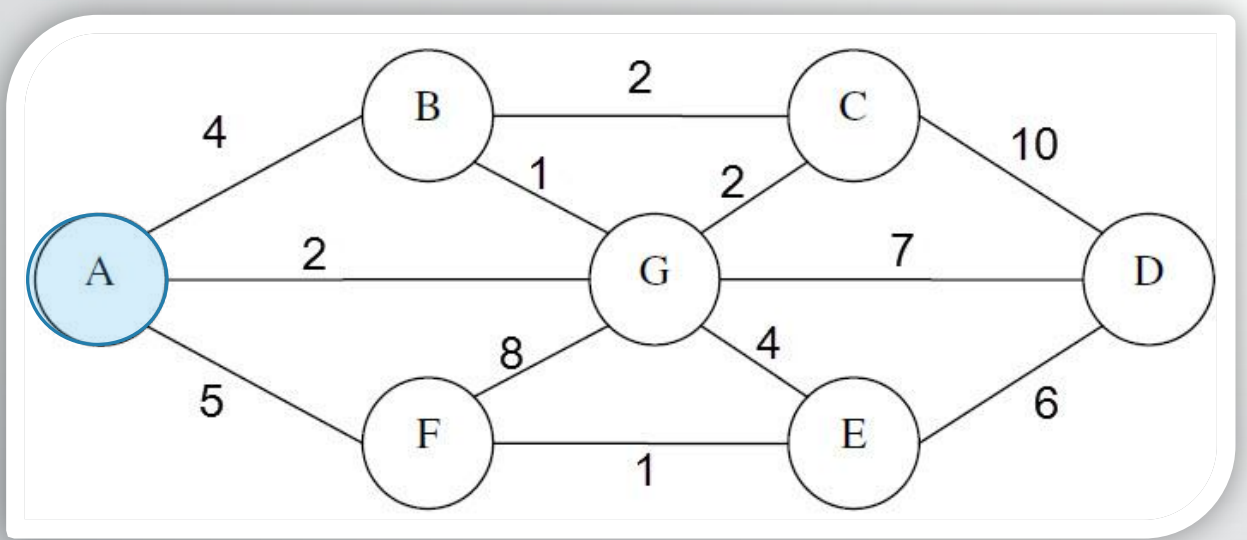
- Start from any node you want , I will start with A , add it to the Q

A

- Result:

Graph Traversal

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

3- pop the front

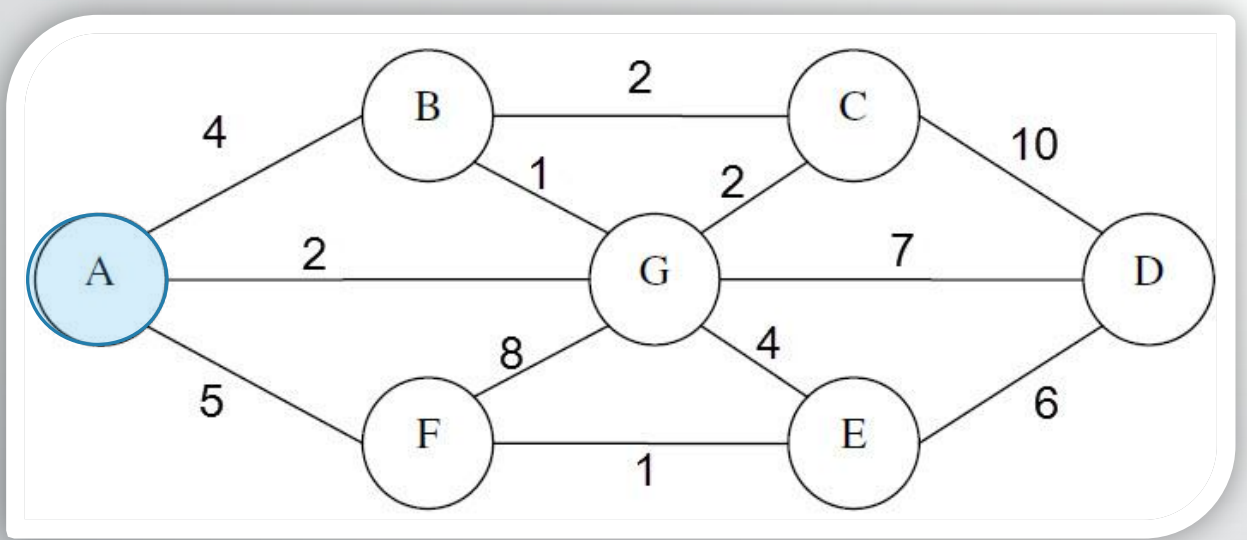
A

✔ Result: A

Graph Traversal

Alphabetic order

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

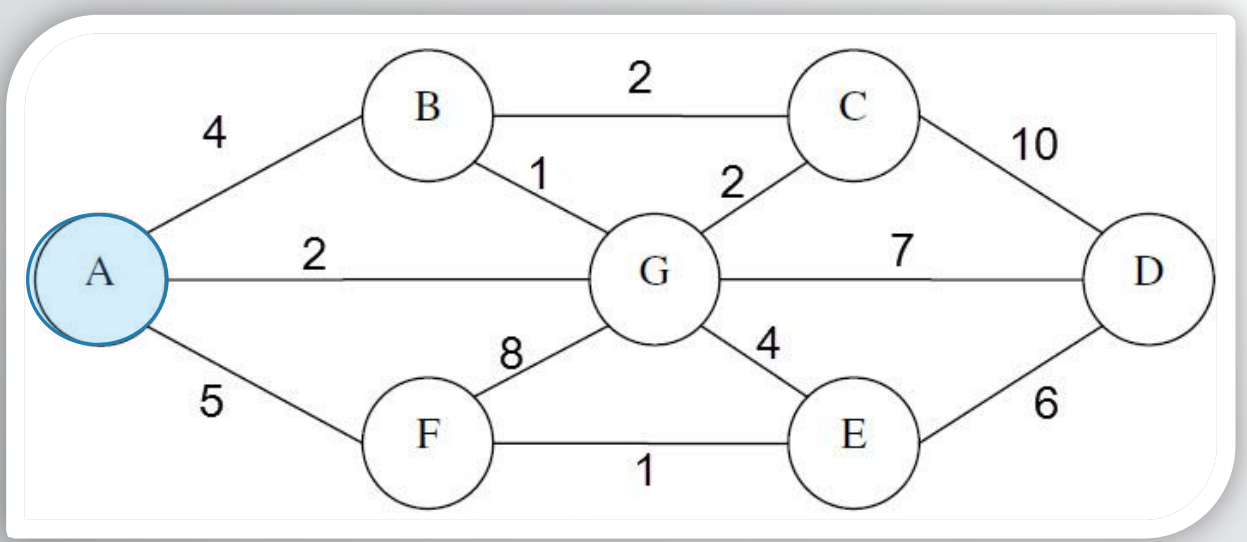
3- pop the front

A B F G

✔ Result: A

Graph Traversal

✔ Breadth First Search (BFS)



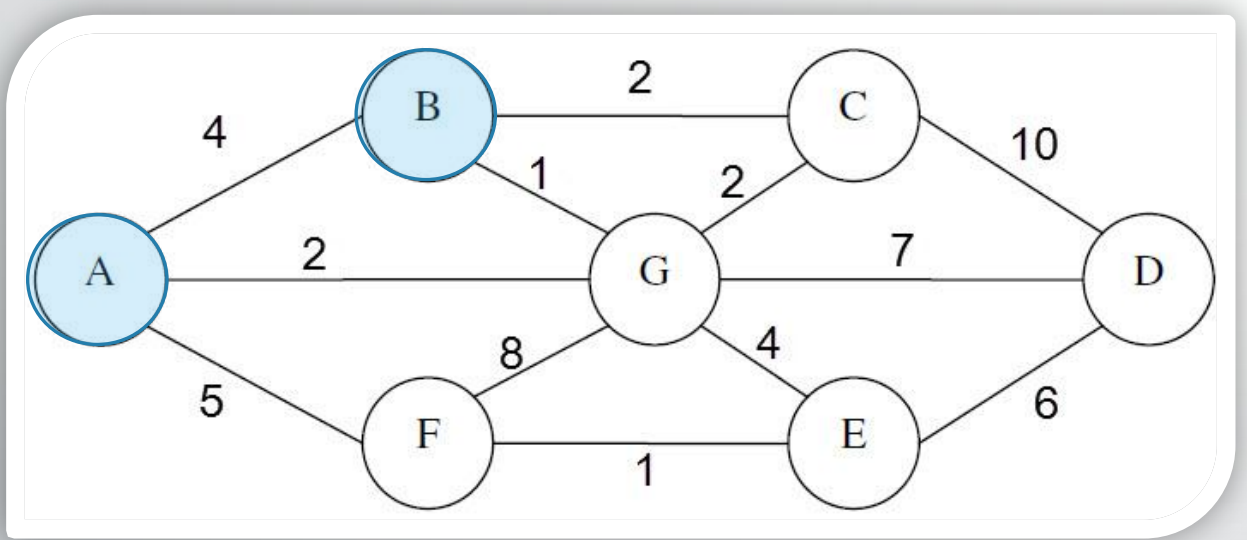
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

B F G

✔ Result: A

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

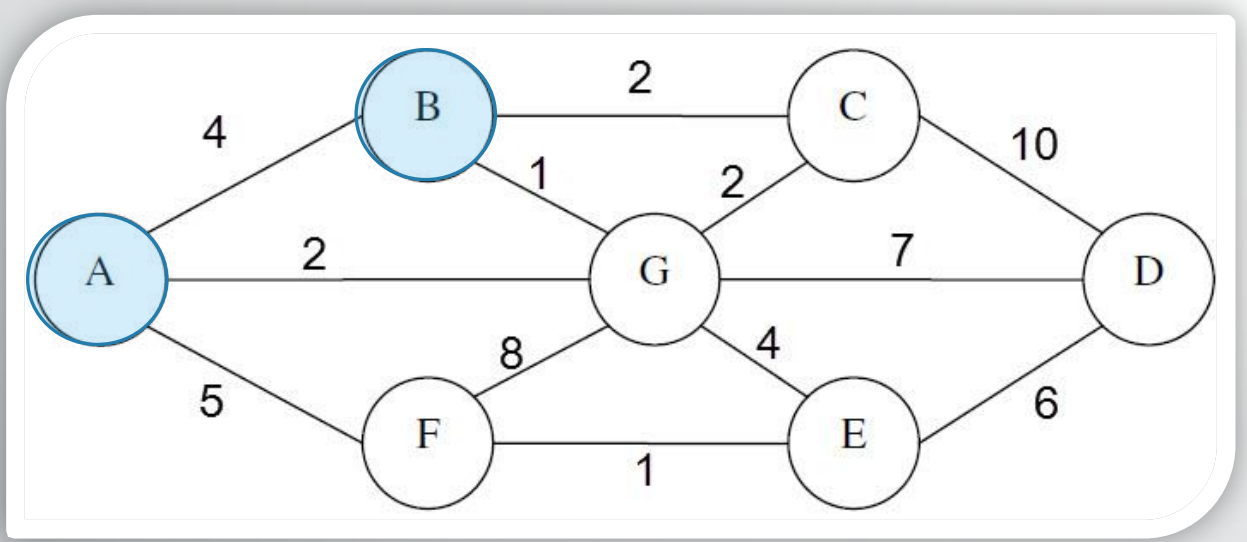
3- pop the front

B F G

✔ Result: A B

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

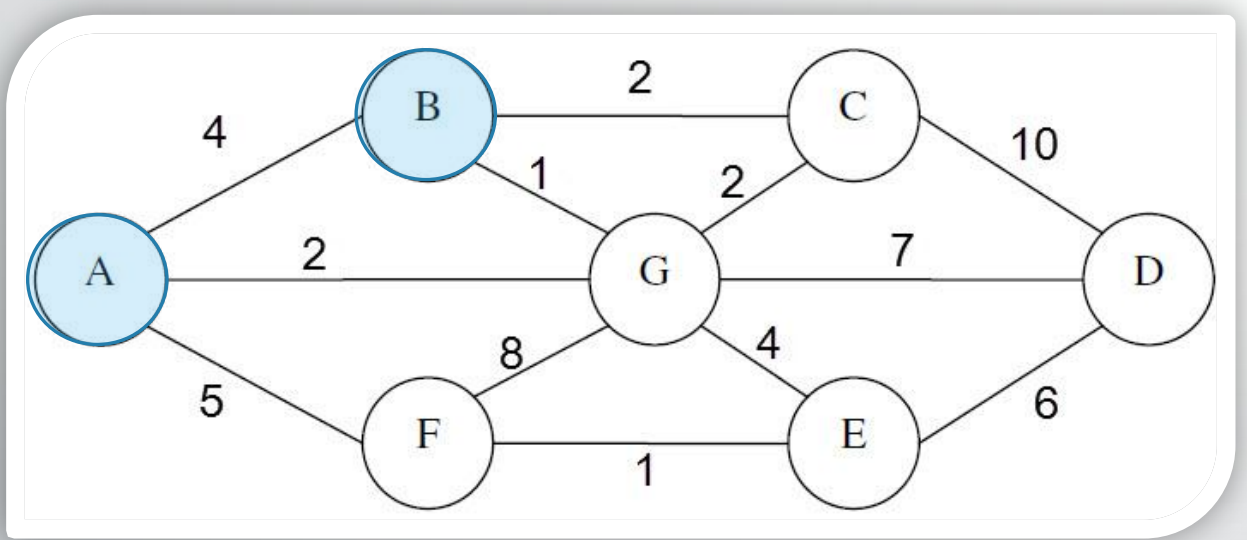
3- pop the front

B F G C

✔ Result: A B

Graph Traversal

✔ Breadth First Search (BFS)



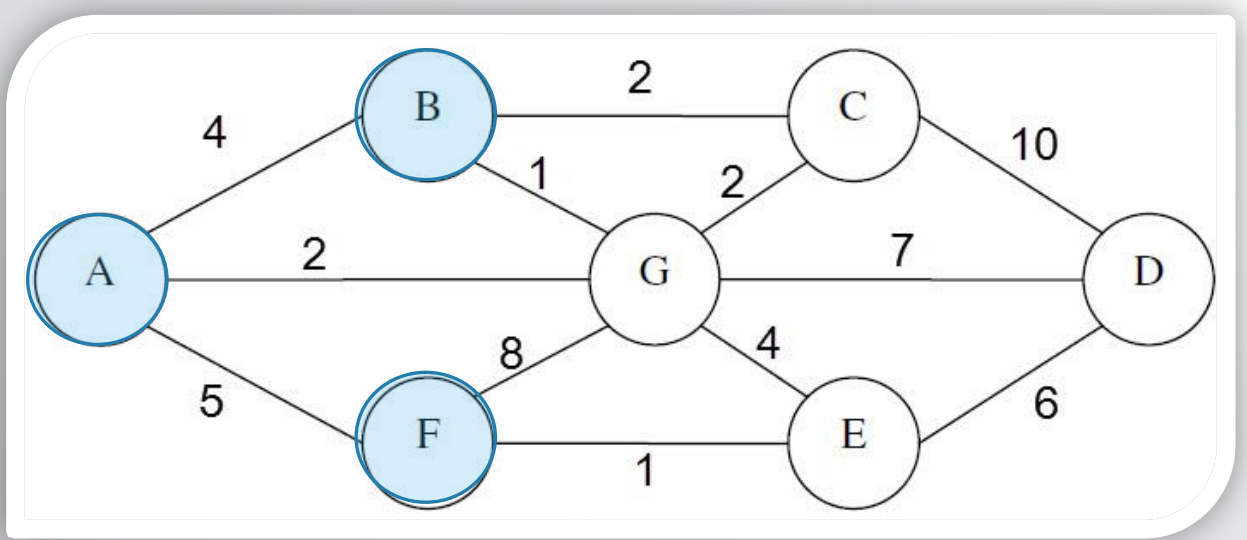
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

F G C

✔ Result: A B

Graph Traversal

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

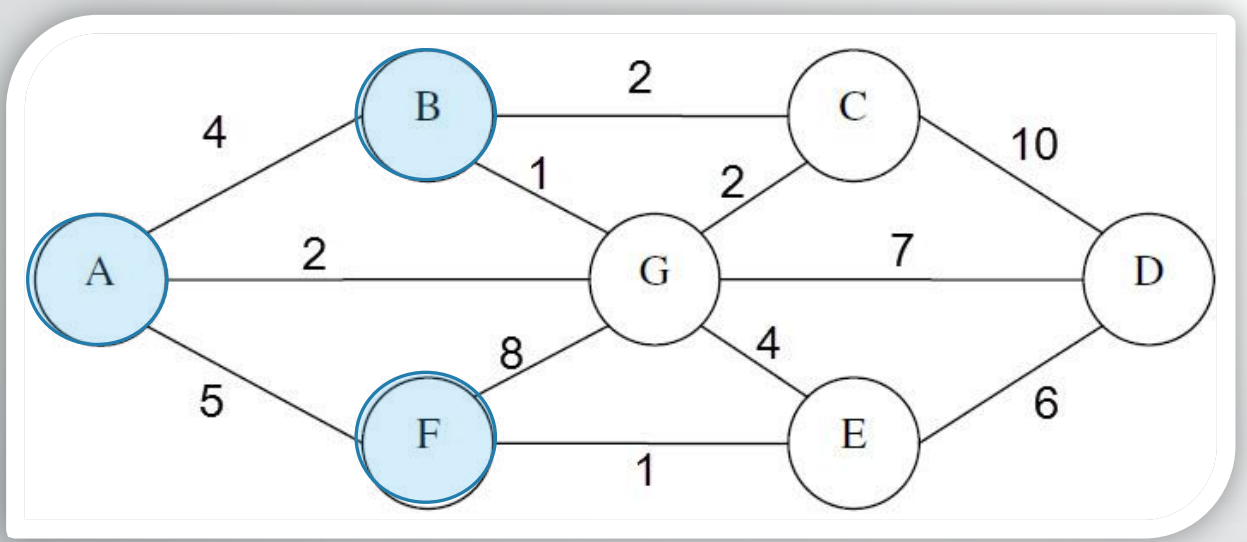
3- pop the front

F G C

✔ Result: A B F

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

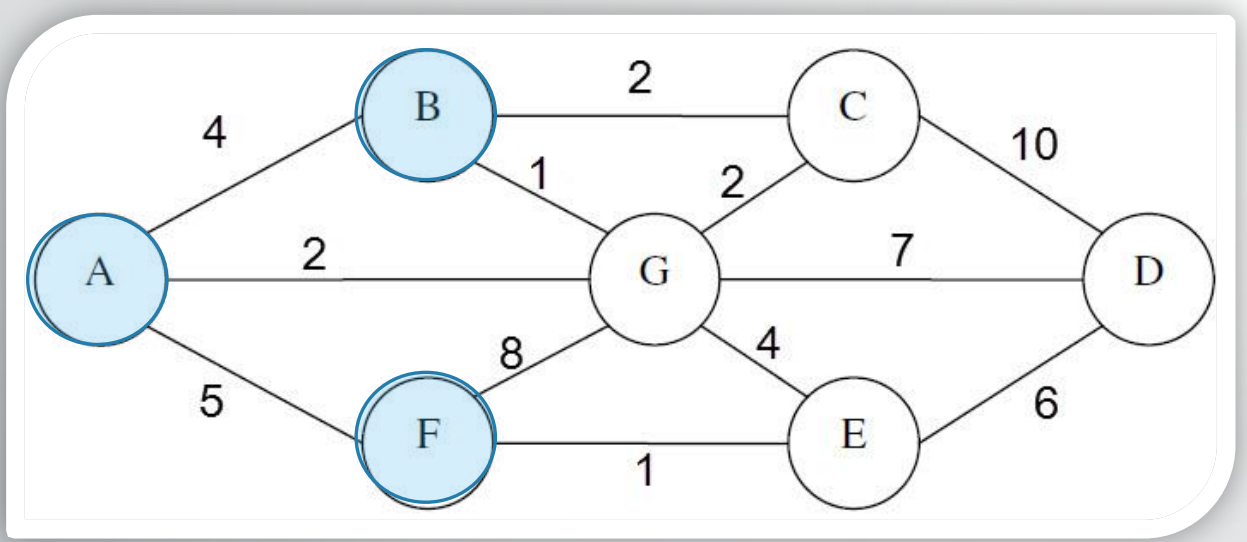
3- pop the front

F G C E

✔ Result: A B F

Graph Traversal

✔ Breadth First Search (BFS)



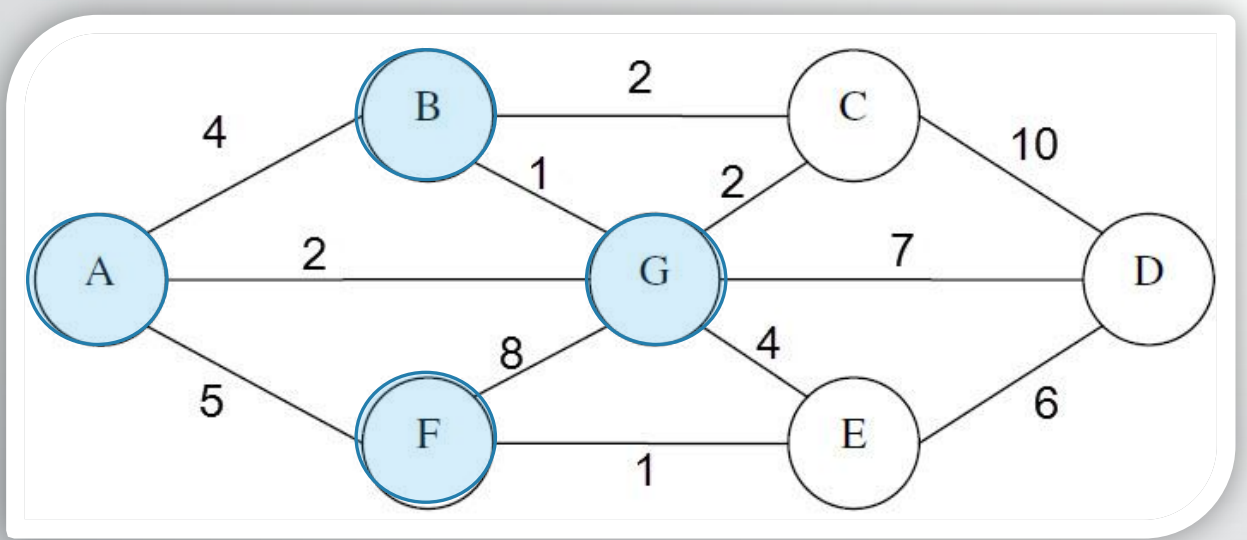
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

G C E

✔ Result: A B F

Graph Traversal

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

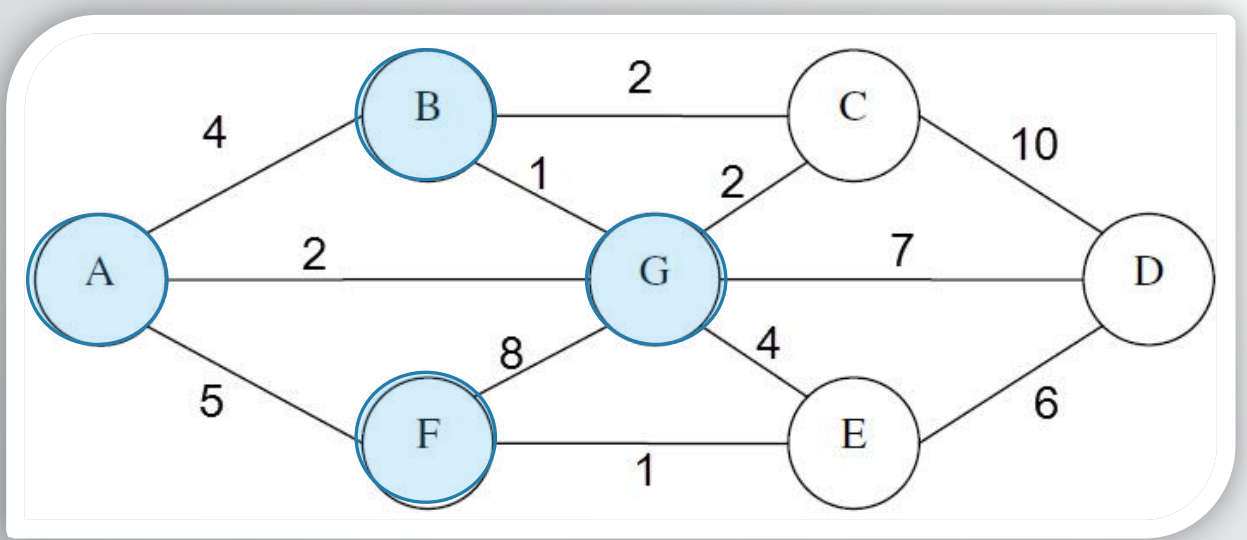
3- pop the front

G C E

✔ Result: A B F G

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

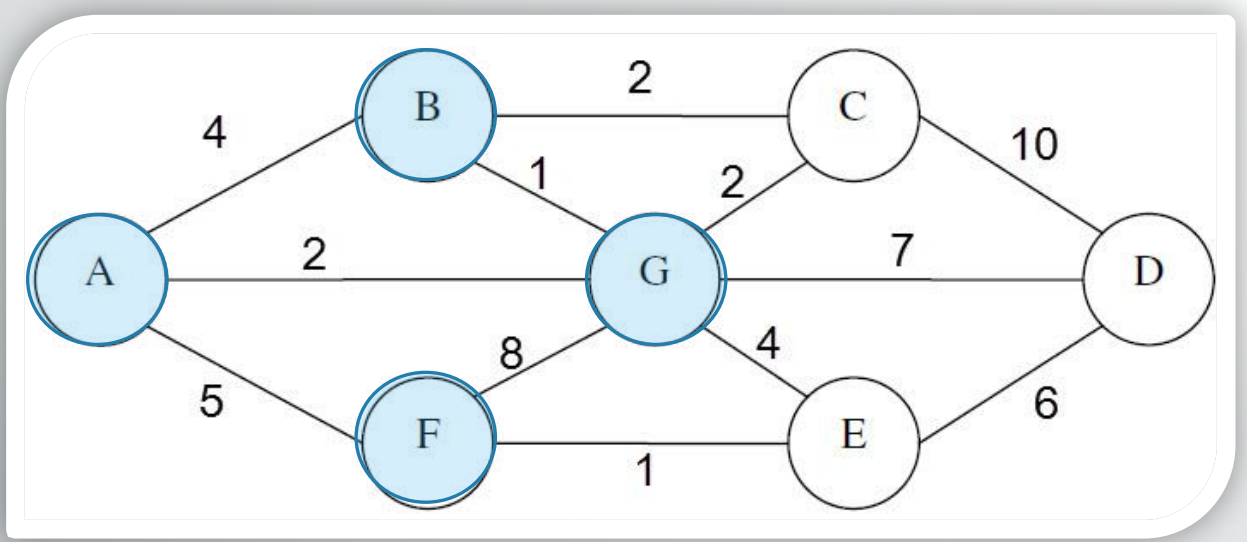
3- pop the front

G C E D

✔ Result: A B F G

Graph Traversal

✔ Breadth First Search (BFS)



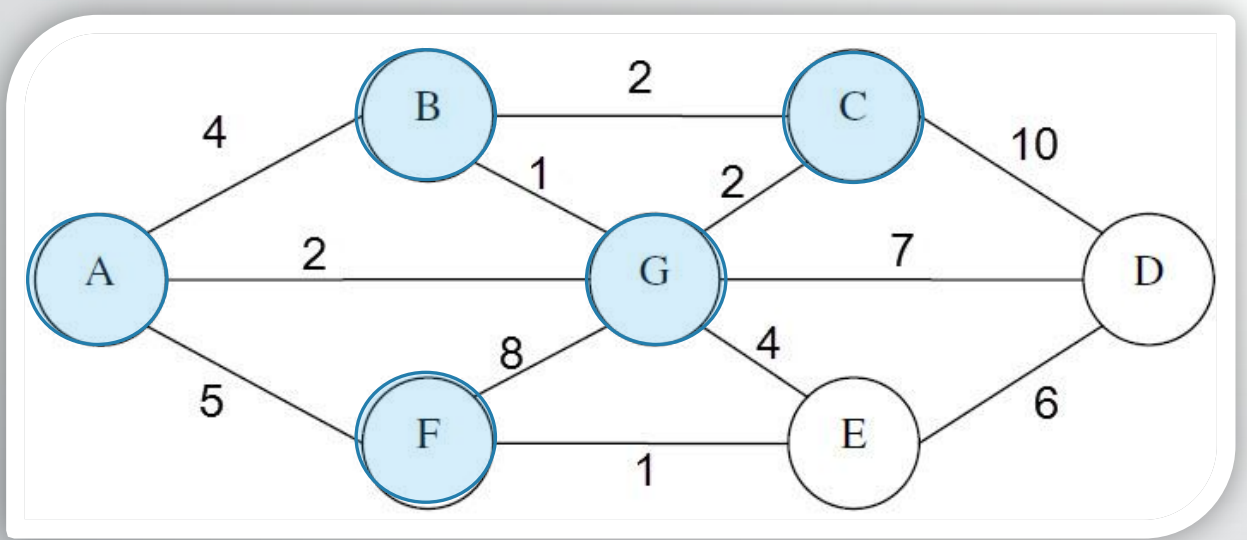
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

C E D

✔ Result: A B F G

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

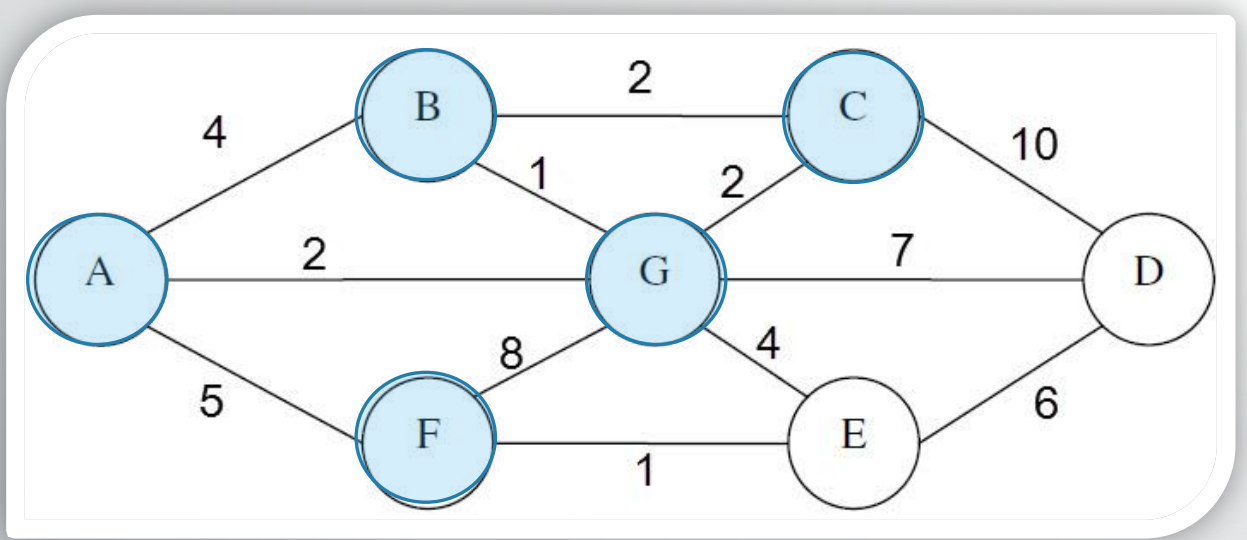
3- pop the front

C E D

✔ Result: A B F G C

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

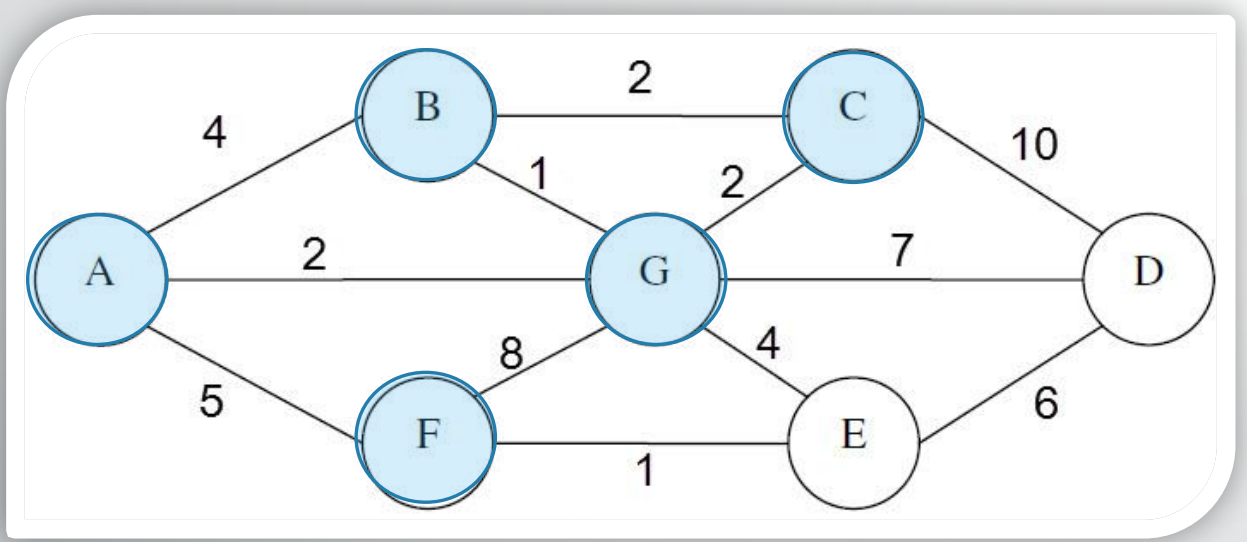
3- pop the front

C E D

✔ Result: A B F G C

Graph Traversal

✔ Breadth First Search (BFS)



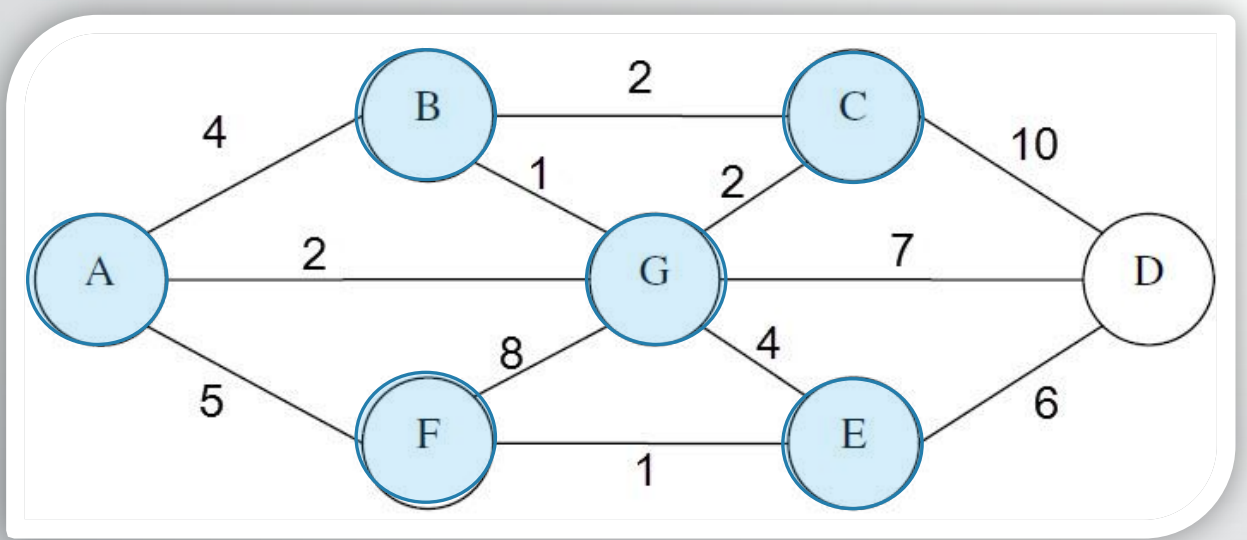
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

E D

✔ Result: A B F G C

Graph Traversal

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

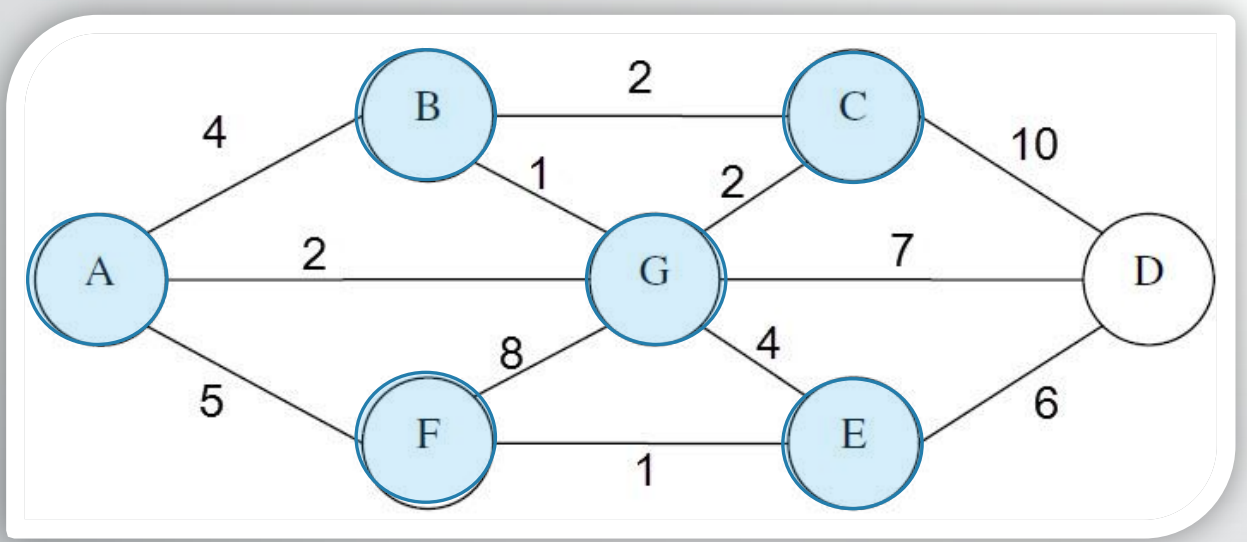
3- pop the front

E D

✔ Result: A B F G C E

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

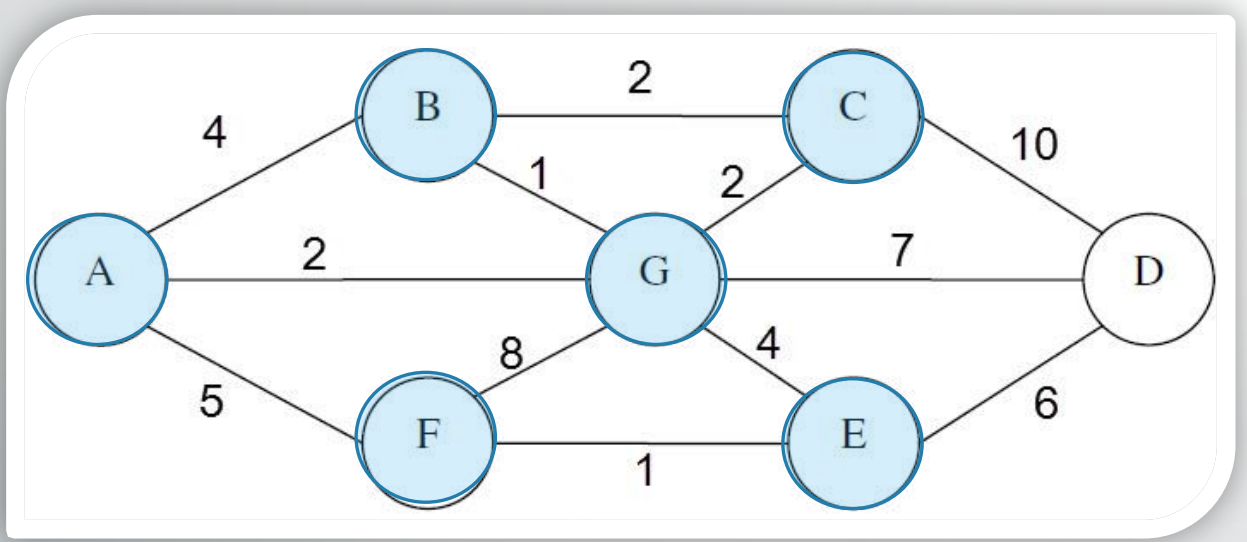
3- pop the front

E D

✔ Result: A B F G C E

Graph Traversal

✔ Breadth First Search (BFS)



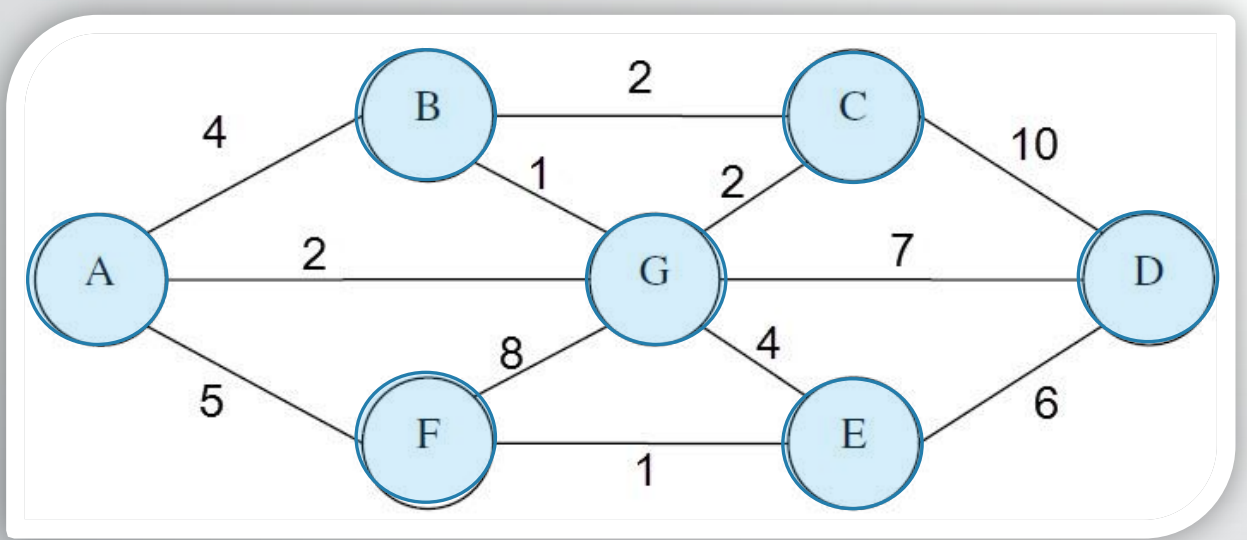
✔ Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

D

✔ Result: A B F G C E

Graph Traversal

✔ Breadth First Search (BFS) ✔



Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

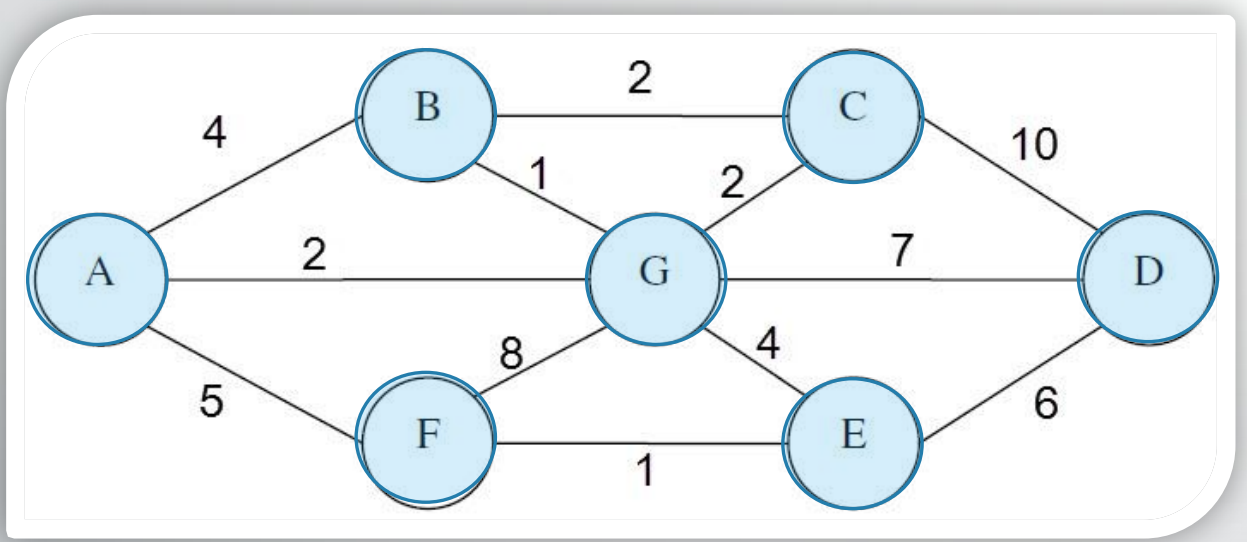
3- pop the front

D

✔ Result: A B F G C E D

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

2- If there's new nodes, add them to the stack

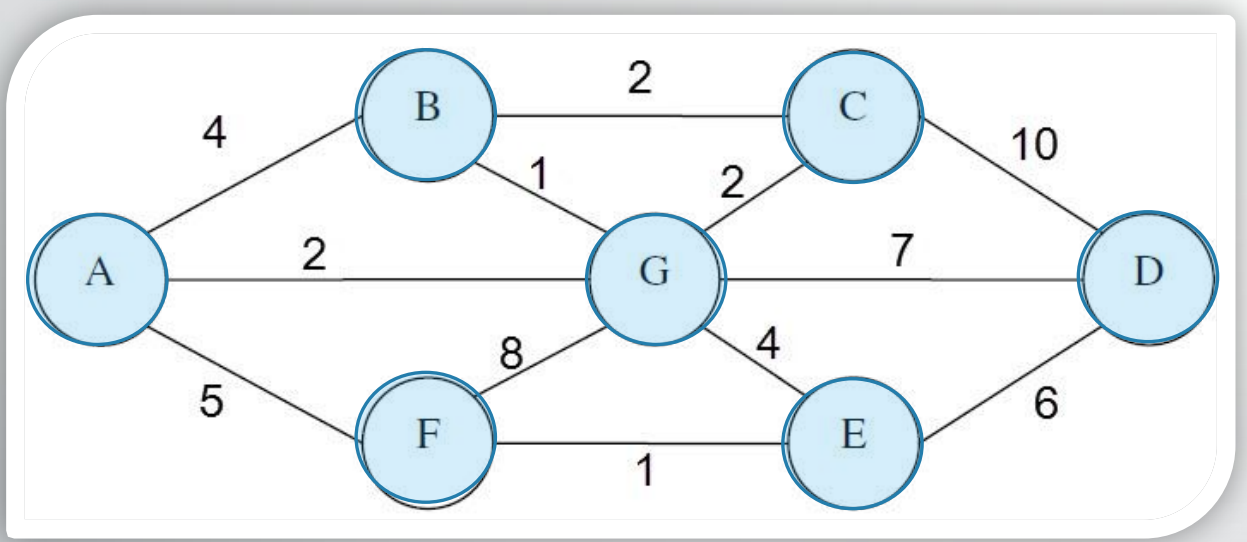
3- pop the front

D

✔ Result: A B F G C E D

Graph Traversal

✔ Breadth First Search (BFS)



✔ Until queue is empty:

1- print the front

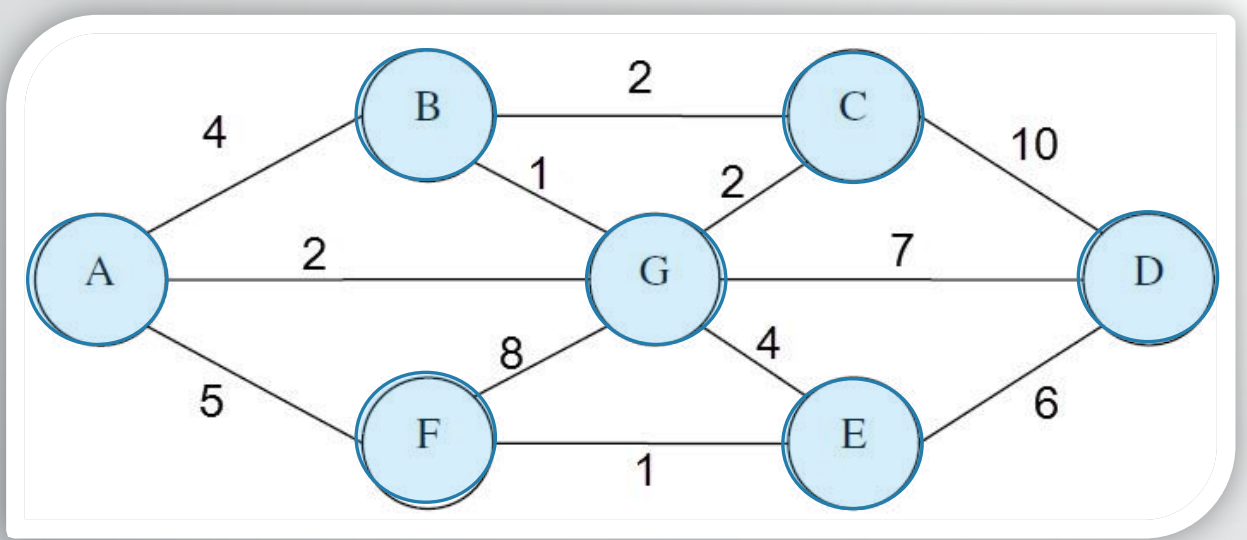
2- If there's new nodes, add them to the stack

3- pop the front

✔ Result: A B F G C E D

Graph Traversal

✔ Breadth First Search (BFS) ✔



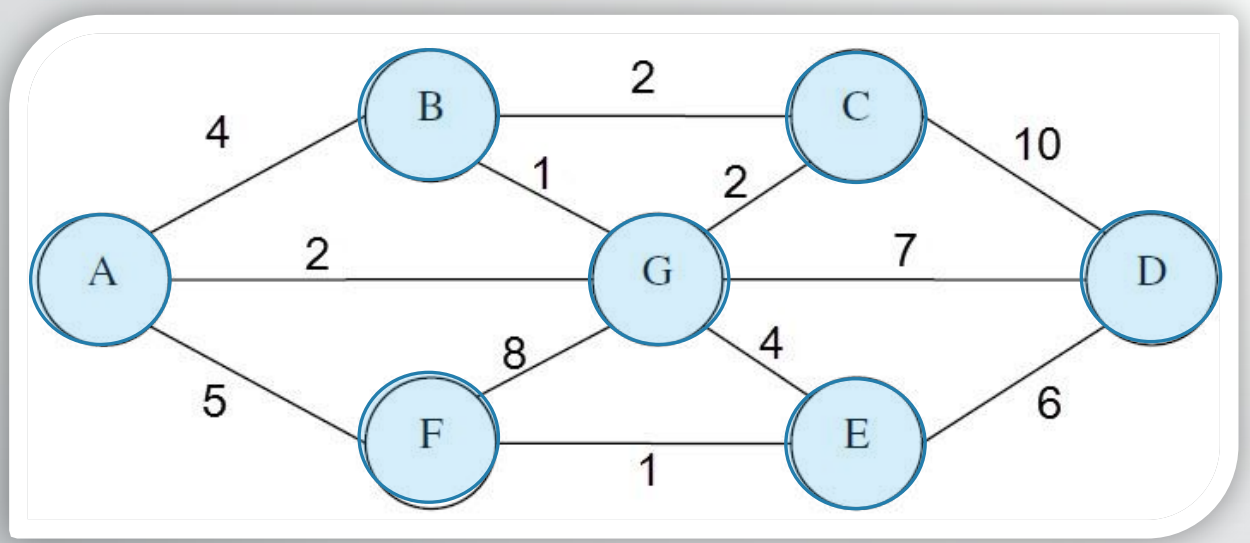
Until queue is empty:
1- print the front
2- If there's new nodes, add them to the stack
3- pop the front

Done ☺

✔ Result: A B F G C E D

Graph Traversal

✔ Breadth First Search (BFS)



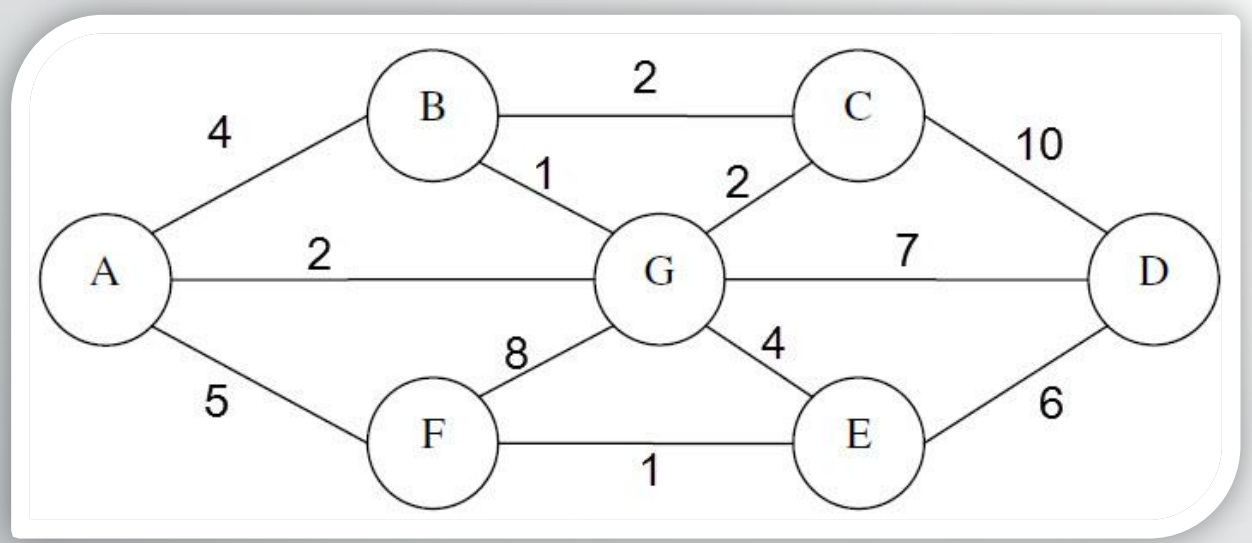
Algorithm :

* Start from any node
* Add it to the Queue
* Until queue is empty:
  1- print the front
  2- If there's new nodes, add them to the queue
  3- pop the front

✔ Result: A B F G C E D

Graph Traversal

- Depth First Search (DFS)

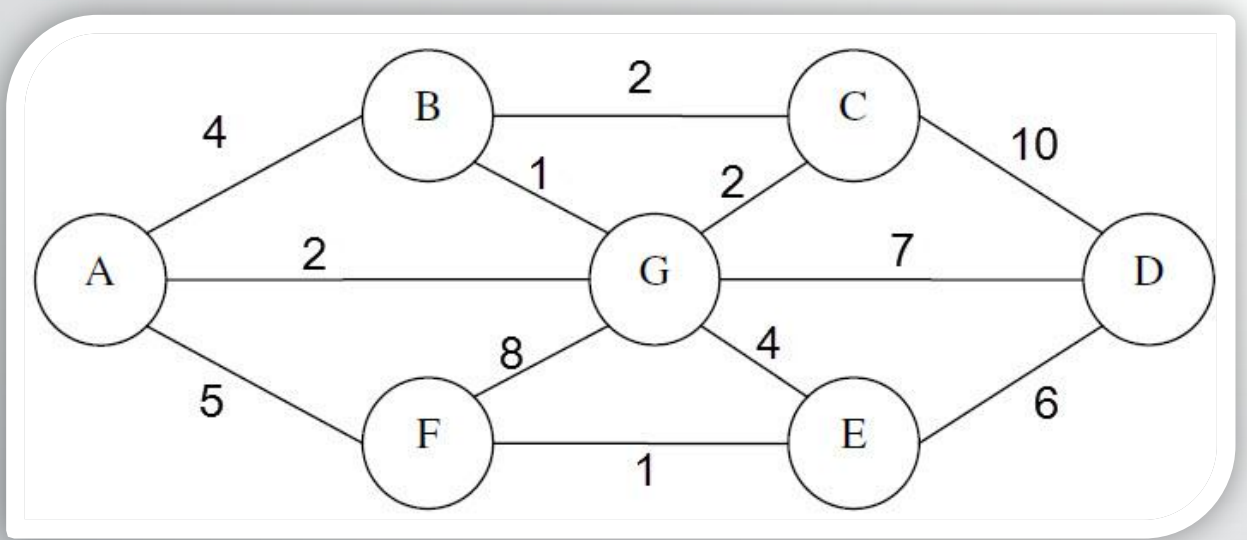


- Instead of using queue , we will use a stack

- Result:

Graph Traversal

- Depth First Search (DFS)



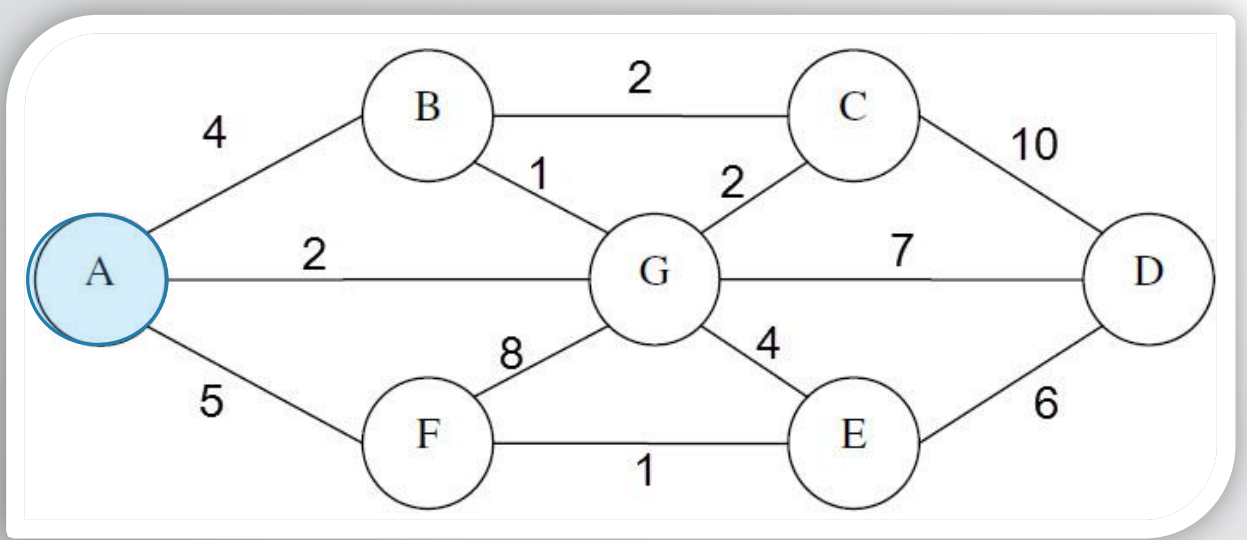
- Start from any node , I will start from A
- Add it to the stack

- Result:

A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

2- If there's new nodes, add one to the stack , else, pop the top of stack

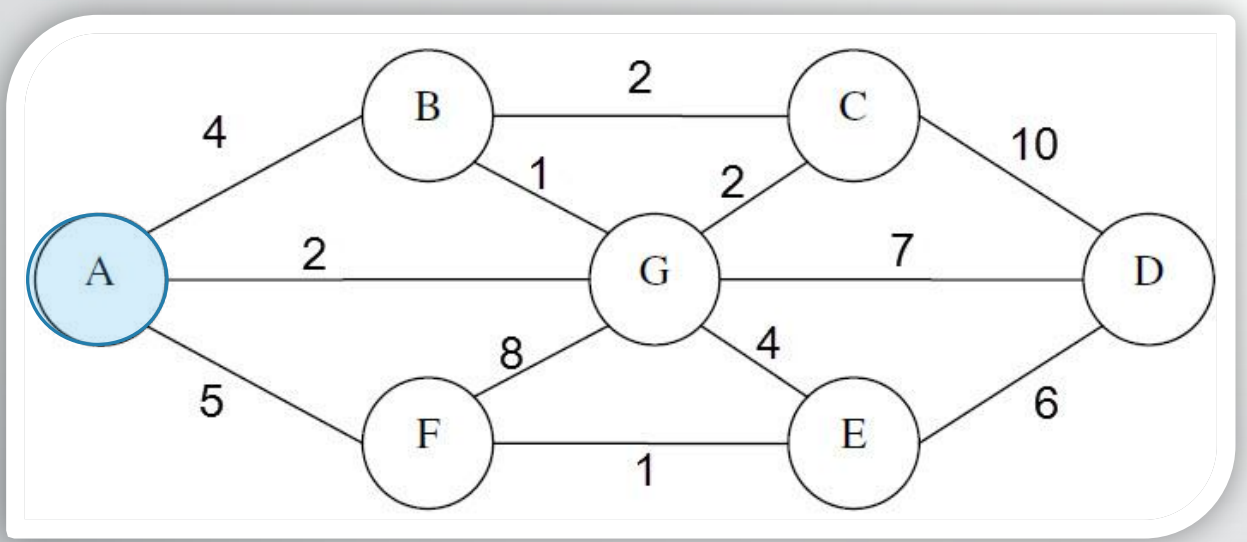
✔ Result: A

A

Graph Traversal

Alphabetic order

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

2- If there's new nodes, add one to the stack ,

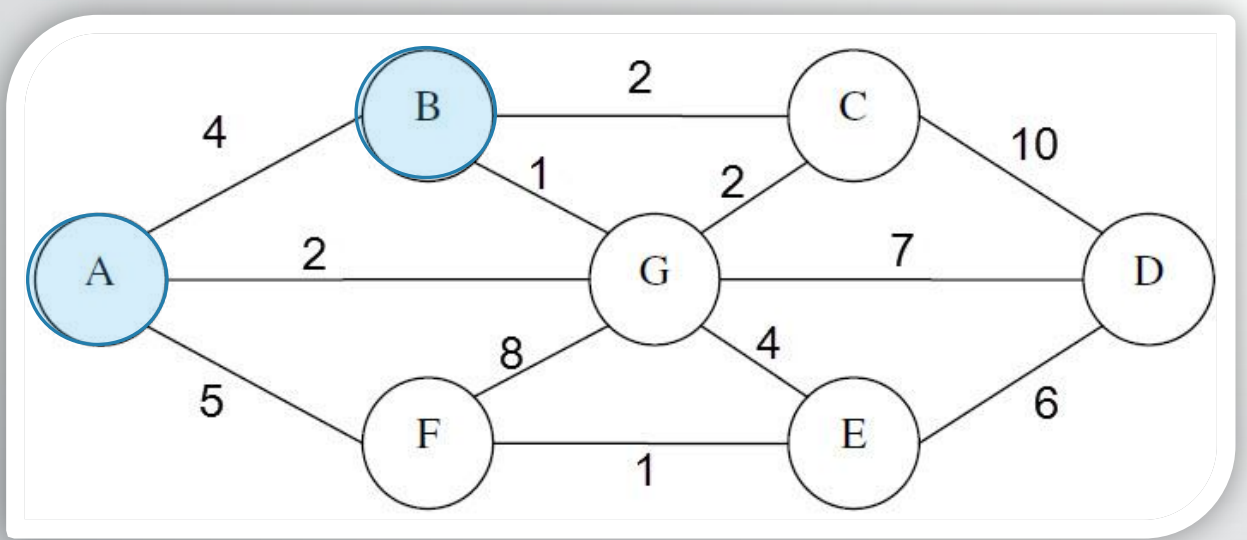
else, pop the top of stack

✔ Result: A

B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

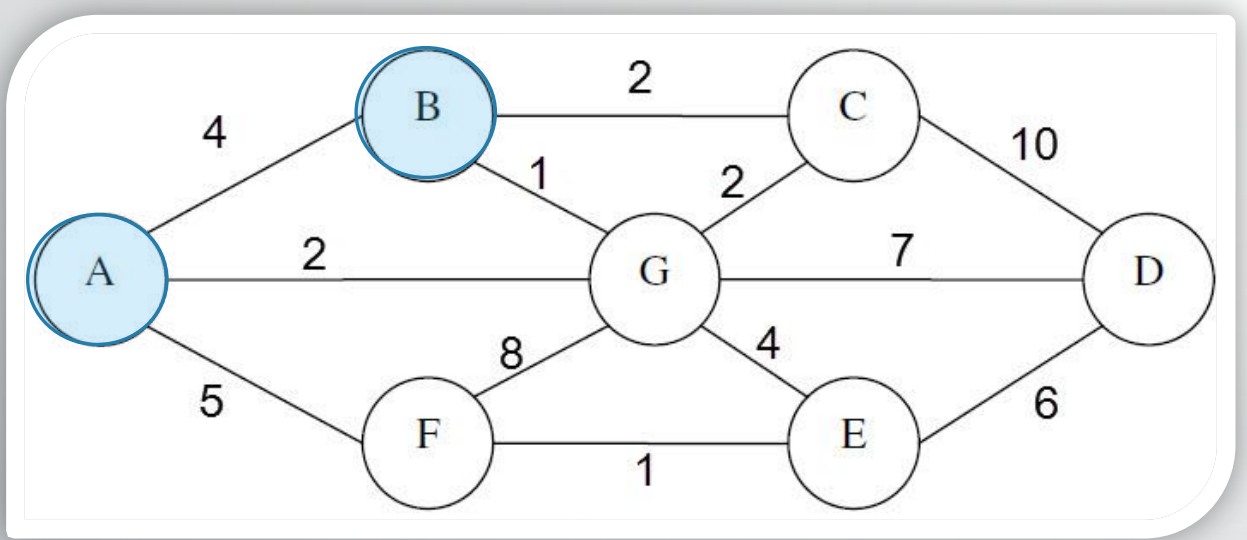
2- If there's new nodes, add one to the stack , else, pop the top of stack

✔ Result: A B

B
A

Graph Traversal

✔ Depth First Search (DFS)✔



✔ Result: A B

Until stack is empty :

1- print the top of stack

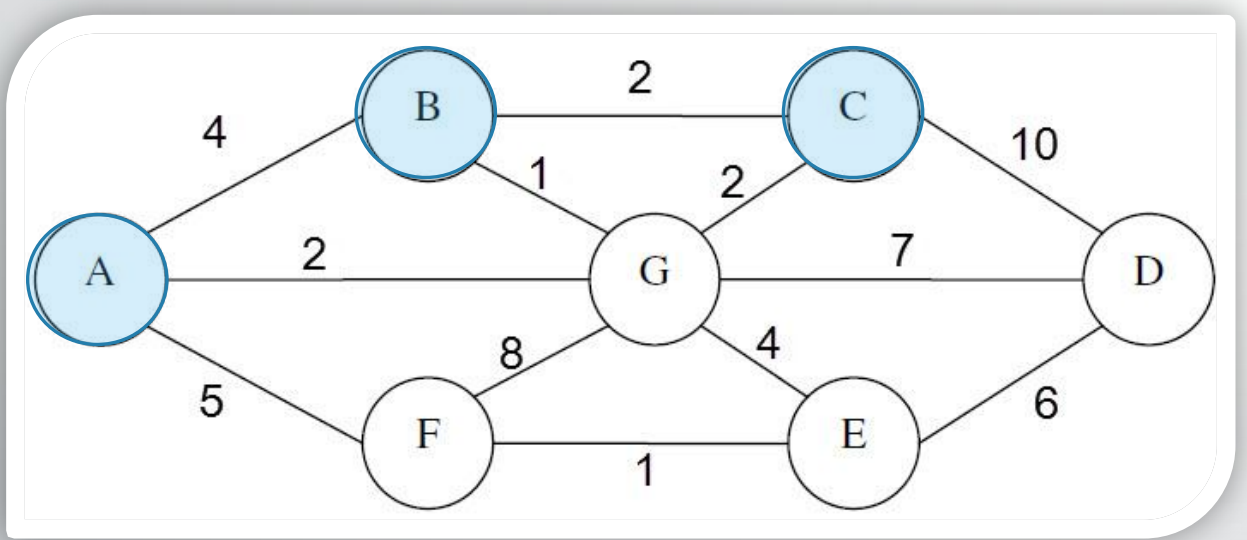
2- If there's new nodes, add one to the stack ,

else, pop the top of stack

C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

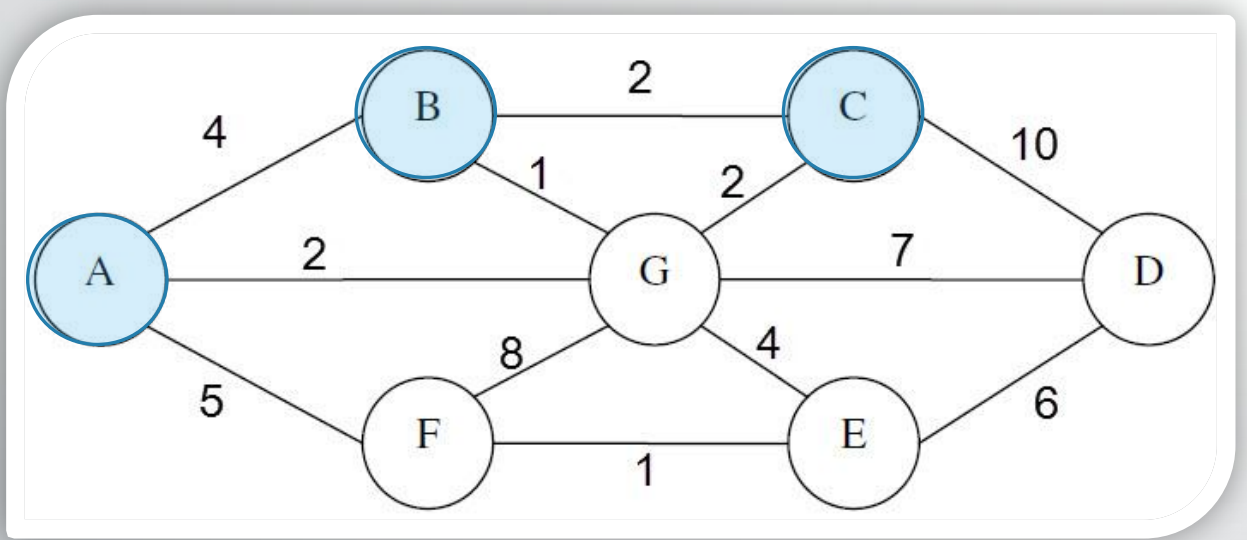
2- If there's new nodes, add one to the stack , else, pop the top of stack

✔ Result: A B C

C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

2- If there's new nodes, add one to the stack ,

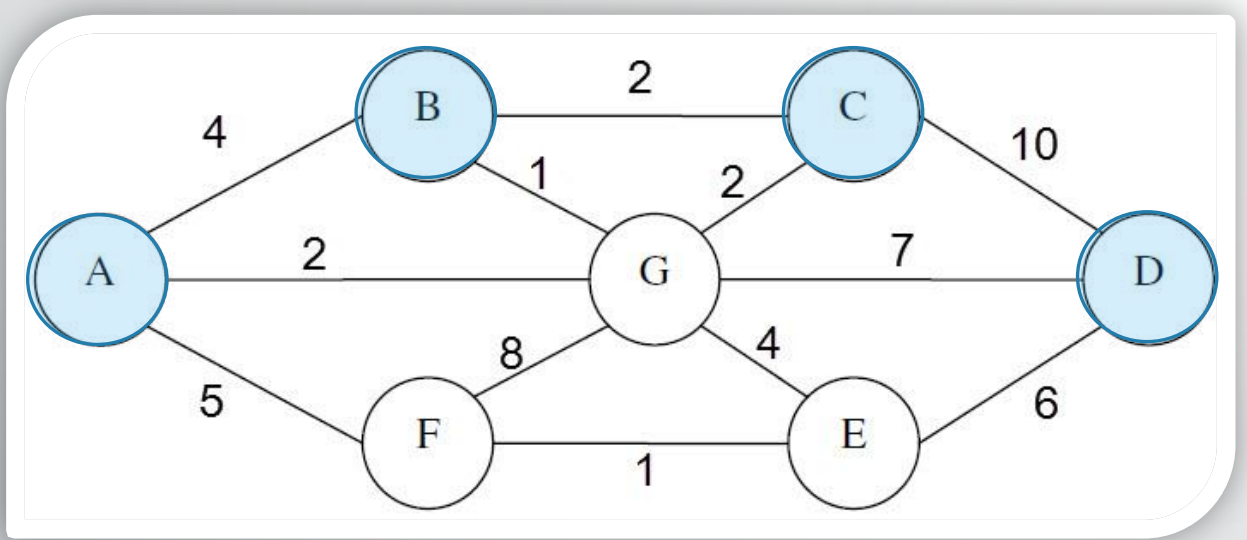
else, pop the top of stack

✔ Result: A B C

D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



✔ Result: A B C D

Until stack is empty :

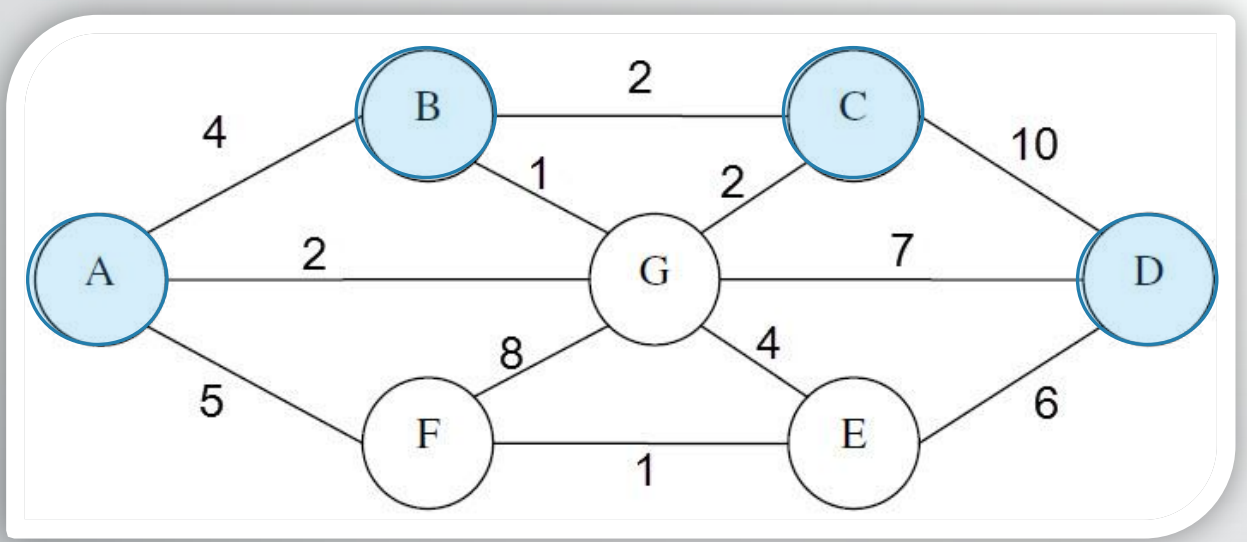
1- print the top of stack

2- If there's new nodes, add one to the stack , else, pop the top of stack

D
C
B
A

Graph Traversal

✔ Depth First Search (DFS)✔



✔ Result: A B C D

Until stack is empty :

1- print the top of stack

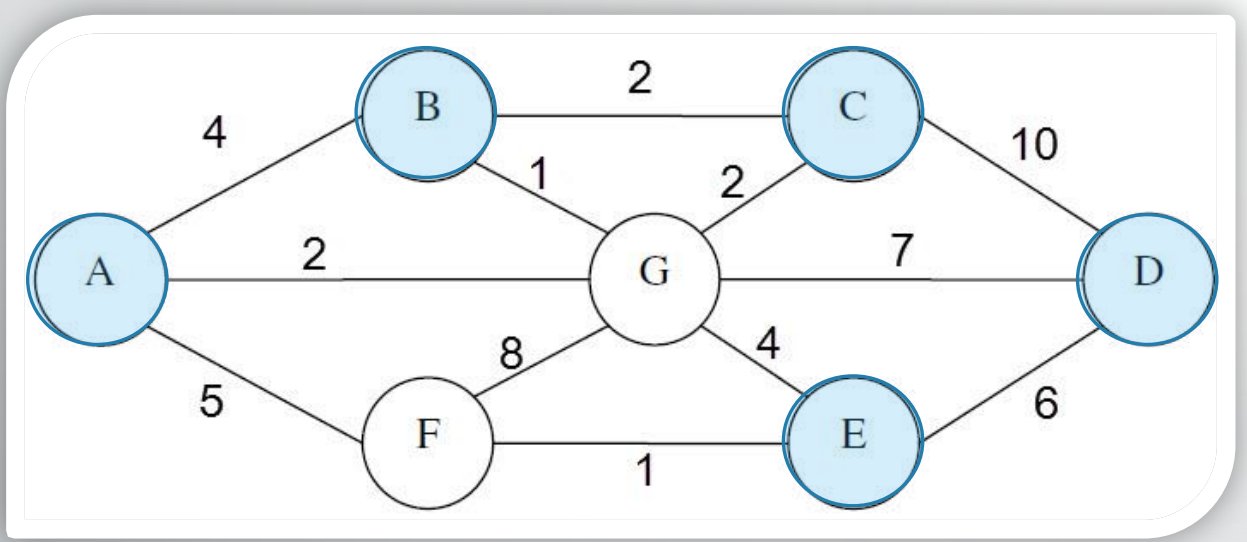
2- If there's new nodes, add one to the stack ,

else, pop the top of stack

E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



✔ Result: A B C D E

Until stack is empty :

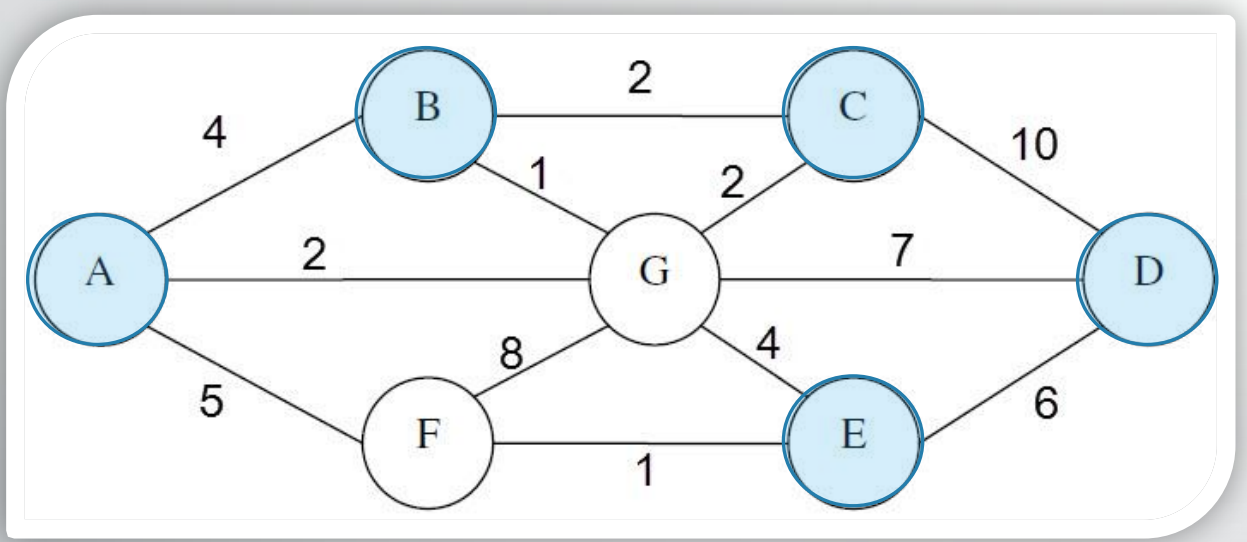
1- print the top of stack

2- If there's new nodes, add one to the stack , else, pop the top of stack

E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS)✔



✔ Result: A B C D E

Until stack is empty :

1- print the top of stack

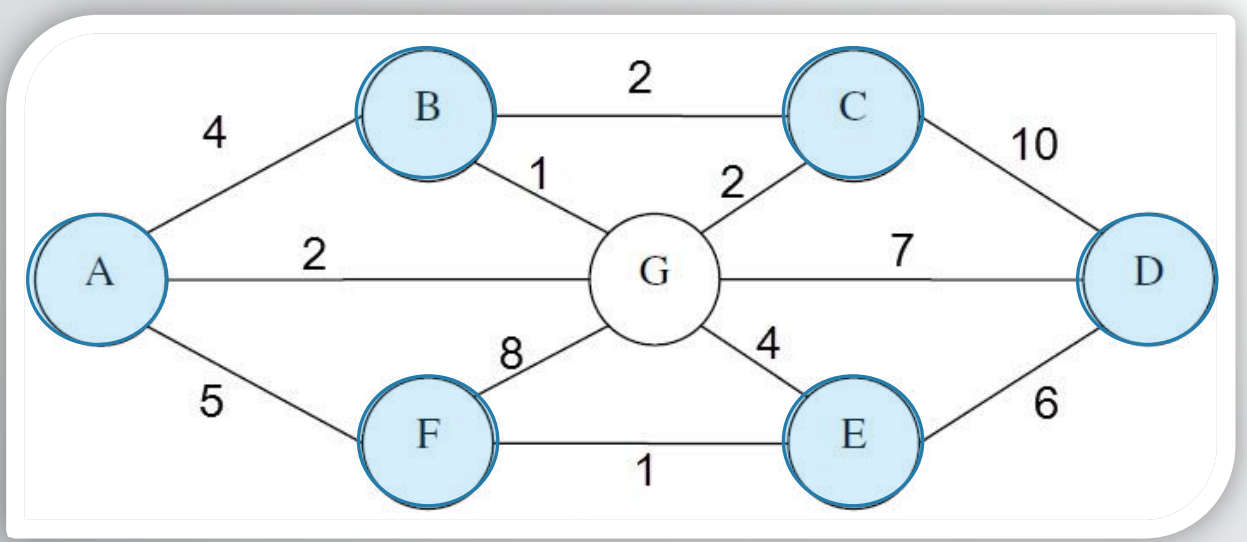
2- If there's new nodes, add one to the stack ,

else, pop the top of stack

F
E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



✔ Result: A B C D E F

Until stack is empty :

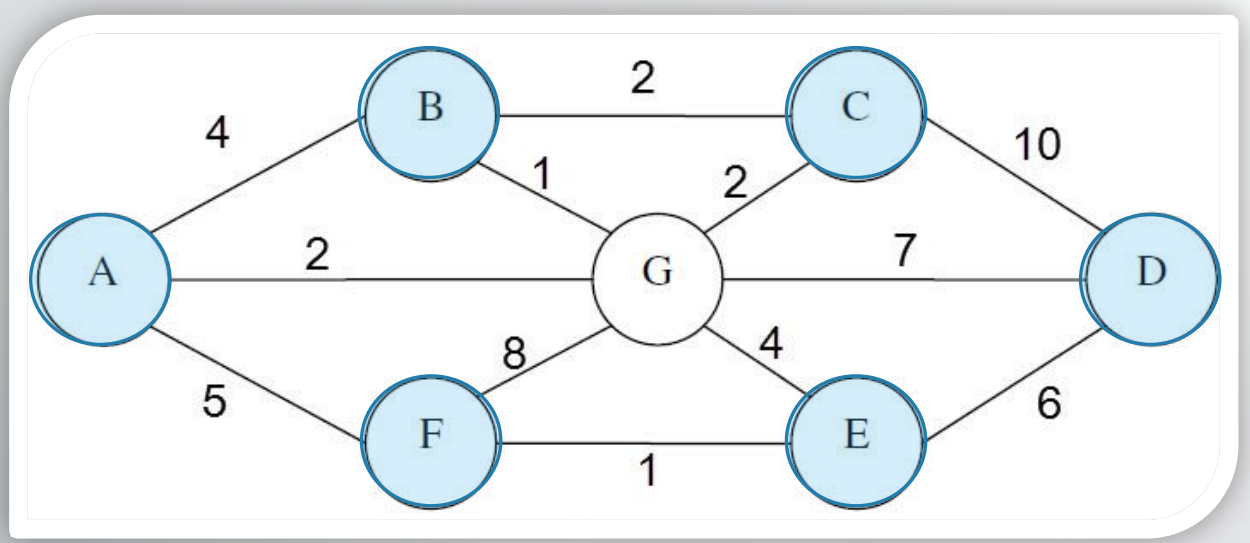
1- print the top of stack

2- If there's new nodes, add one to the stack , else, pop the top of stack

F
E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS)✔



✔ Result: A B C D E F

Until stack is empty :

1- print the top of stack

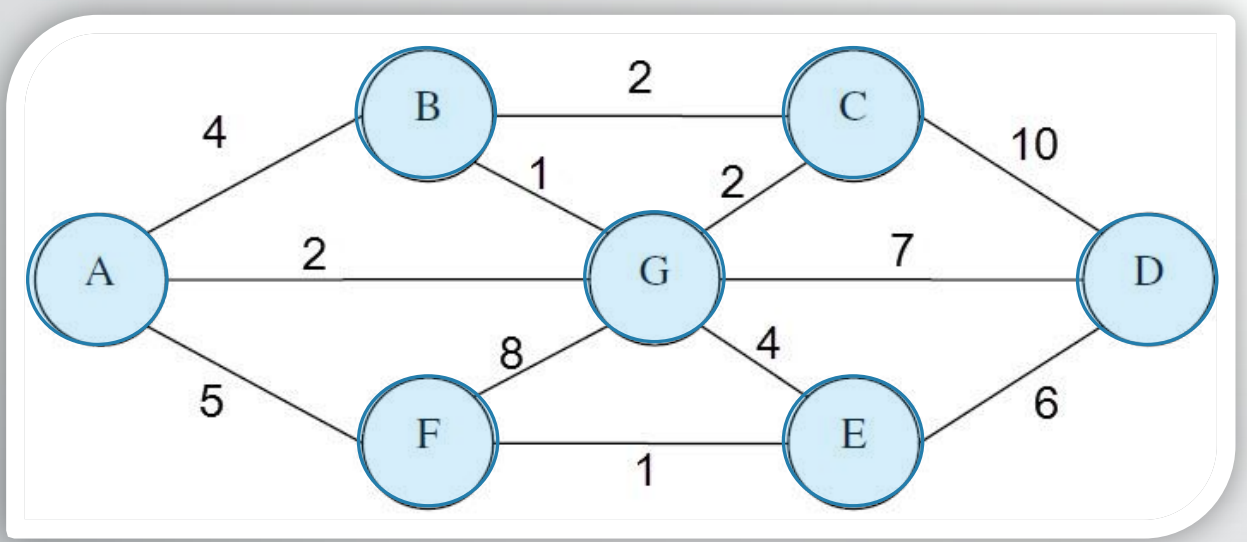
2- If there's new nodes, add one to the stack ,

else, pop the top of stack

G
F
E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

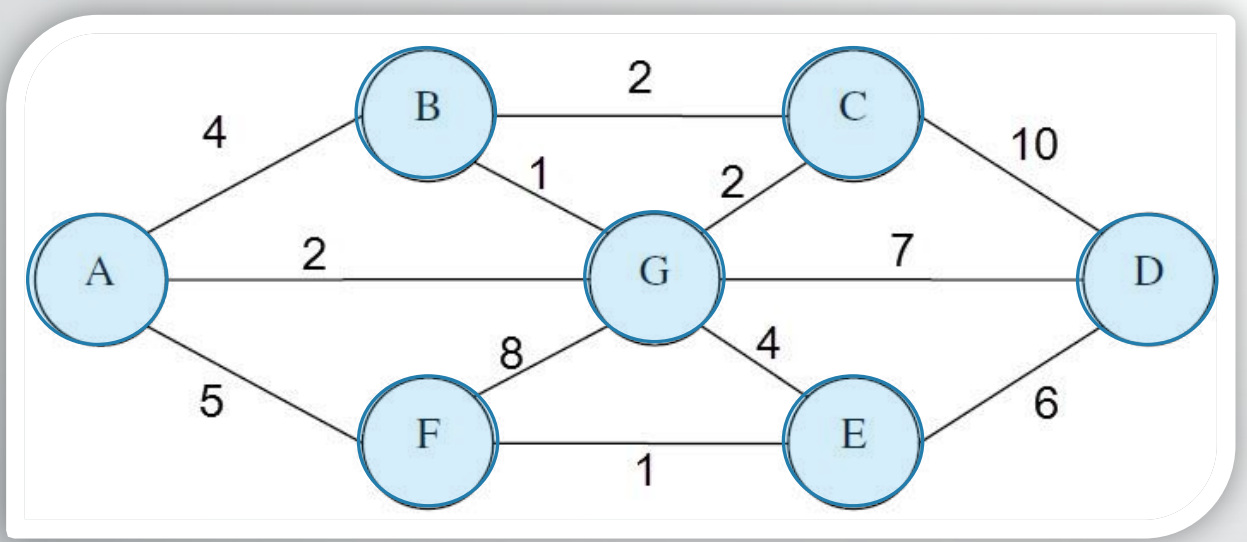
2- If there's new nodes, add one to the stack , else, pop the top of stack

G
F
E
D
C
B
A

✔ Result: A B C D E F G

Graph Traversal

✔ Depth First Search (DFS) ✔



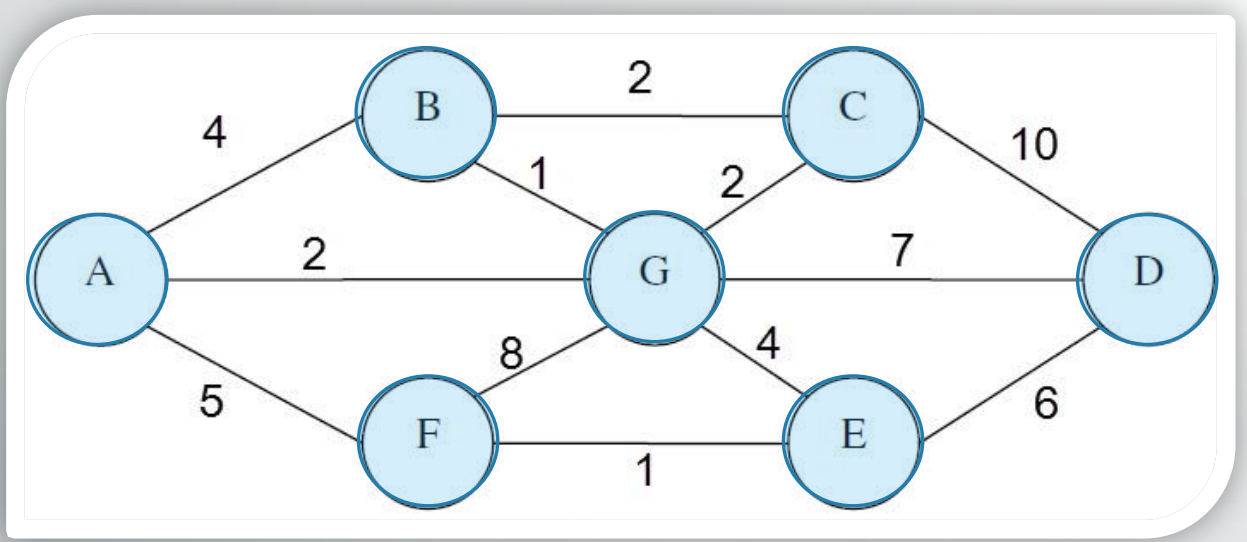
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

F
E
D
C
B
A

✔ Result: A B C D E F G

Graph Traversal

✔ Depth First Search (DFS) ✔



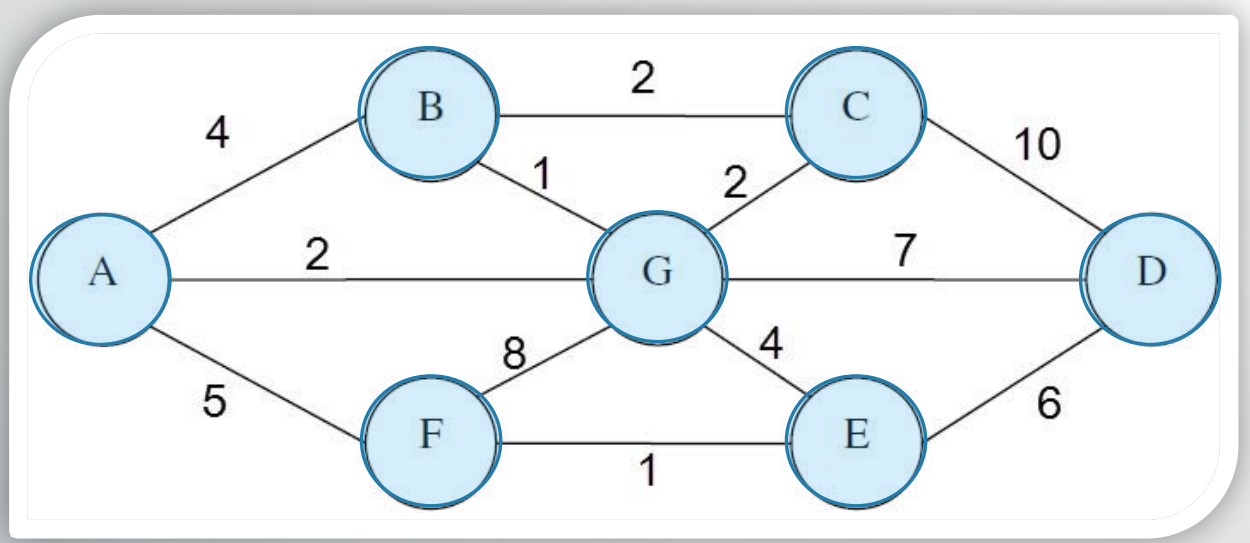
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

✔ Result: A B C D E F G

E
D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



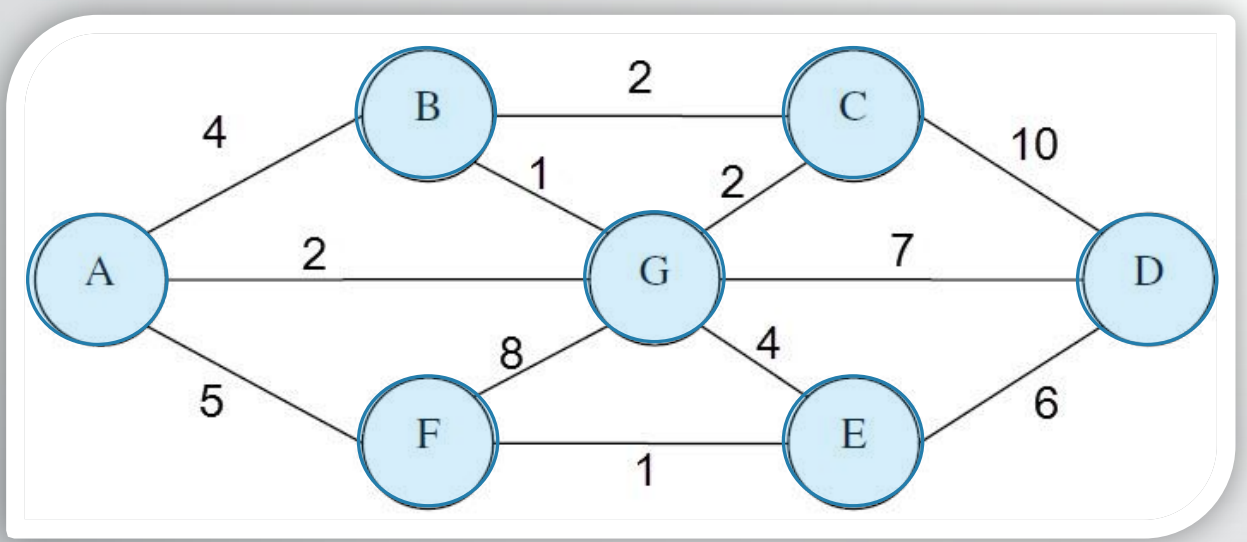
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

✔ Result: A B C D E F G

D
C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



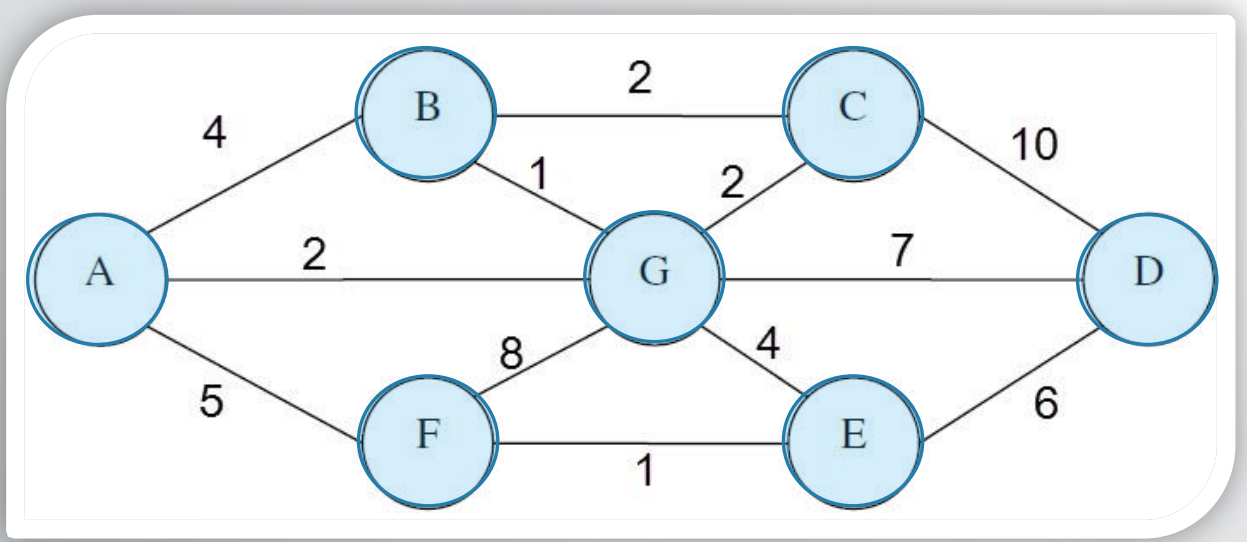
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

✔ Result: A B C D E F G

C
B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



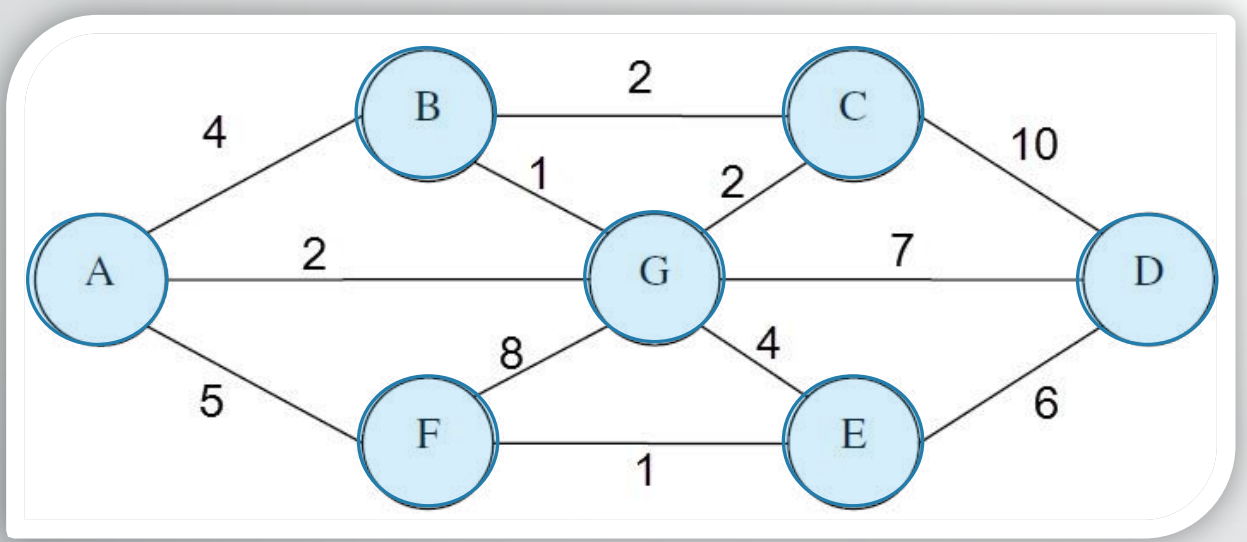
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

✔ Result: A B C D E F G

B
A

Graph Traversal

✔ Depth First Search (DFS) ✔



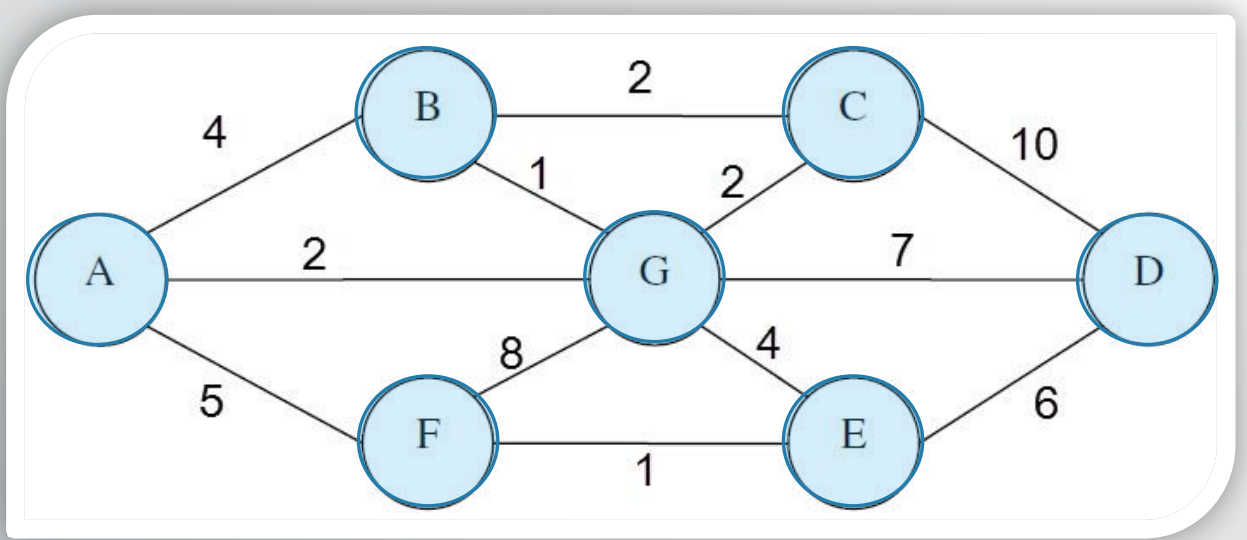
Until stack is empty :
1- print the top of stack
2- If there's new nodes, add one to the stack ,
else, pop the top of stack

✔ Result: A B C D E F G

A

Graph Traversal

✔ Depth First Search (DFS) ✔



Until stack is empty :

1- print the top of stack

2- If there's new nodes, add one to the stack , else, pop the top of stack

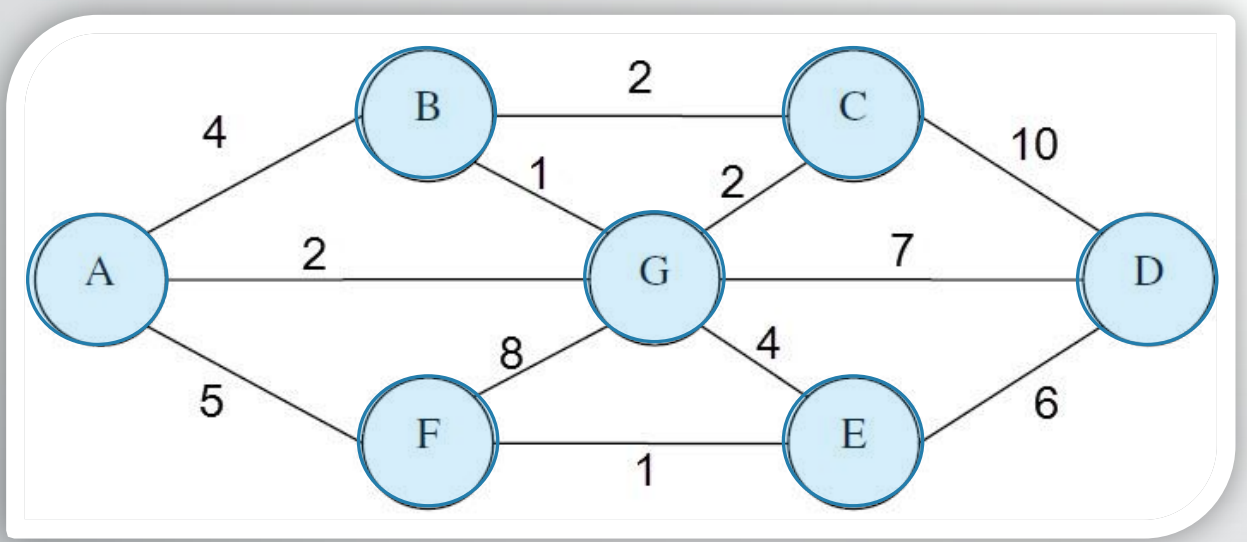
Done ☺

✔ Result: A B C D E F G

Graph Traversal

Print will work only if stack is not empty

✔ Depth First Search (DFS)



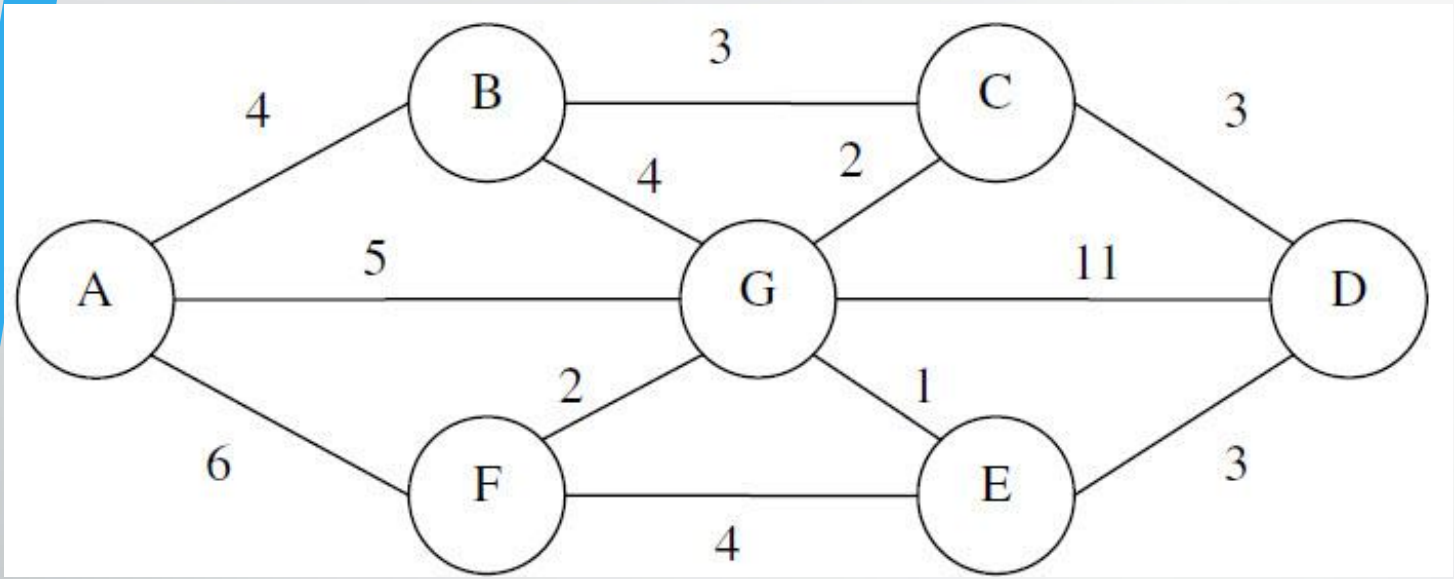
Algorithm :

* Start from any node
* Add it to the Stack
* Until stack is empty:
  1- print the top of S
  2- If there's new nodes, add one to the S based on the alphabetic order, else,
  pop the top of S

✔ Result: A B C D E F G

Extra Question

✔ **Solve The Following:**



For the above graph:

1. Find the corresponding adjacency matrix.
2. Find the corresponding dynamic (linked list) representation.
3. Find the breadth first search and depth first search traversals.

END of Section

Thank You